

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования

ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ СИСТЕМ
УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)

Кафедра компьютерных систем в управлении и проектировании (КСУП)

В.С. Швоев, А.А. Калентьев, Н.А. Набережнев

ЛАБОРАТОРНЫЕ РАБОТЫ ПО ДИСЦИПЛИНЕ ОСНОВЫ РАЗРАБОТКИ
СИСТЕМ АВТОМАТИЗИРОВАННОГО ПРОЕКТИРОВАНИЯ (ОРСАПР)
(учебно-методическое пособие для бакалавров технических вузов)

Томск 2024

УДК 004.422.832

ББК 74.46

Шв 33

Рецензент:

Д.А. Жабин, старший разработчик информационных систем в ООО «Озон Технологии», к.т.н.

А.А. Самуилов, генеральный директор ООО «Арвью», к.т.н.

**Швоев Владимир Сергеевич, Калентьев Алексей Анатольевич,
Набережнев Николай Александрович**

Шв 33 Лабораторные работы по дисциплине основы разработки систем автоматизированного проектирования (ОРСАПР) (учебно-методическое пособие для бакалавров технических вузов) : учебное методическое пособие / В.С. Швоев, А.А. Калентьев, Н.А. Набережнев – Томск: Томск. Гос. ун-т систем упр. и радиоэлектроники, 2024. – 89 с.

Данное учебное пособие составлено с учетом всех требований Федерального государственного образовательного стандарта высшего образования (ФГОС ВО) и предназначено для совершенствования навыков разработки программного обеспечения (ПО) в области разработки систем автоматизированного проектирования (САПР). В пособии рассматриваются полный жизненный цикл разработки ПО от составления технического задания (ТЗ) до написания пояснительной записки (ПЗ). В пособии отображены использования различных инструментов проектирования и разработки ПО. Кроме того, рассматриваются различные архитектурные подходы разработки ПО.

Пособие предназначено для бакалавров технических направлений подготовки.

Одобрено на заседании кафедры КСУП, протокол № 9 от 26.04.2024 г

УДК 004.422.832

ББК 74.46

© Швоев В.С., 2024

© Калентьев А.А., 2024

© Набережнев Н.А., 2024

© Томск. гос. ун-т систем упр. и радиоэлектроники, 2024

Содержание

Сокращения.....	4
Лабораторная 1. Выбор предмета проектирования.	5
Лабораторная 2. Техническое задание.	9
Лабораторная 3. Проект системы.	12
1 Титульный лист	12
2 Описание САПР	14
2.1 Информация о выбранной САПР	14
2.2 Описание API.....	14
2.3 Обзор аналогов плагина	15
3 Описание предмета проектирования	18
4 Проект системы	18
4.1 Диаграмма классов.....	18
4.2 Макеты пользовательского интерфейса	28
5 Список используемых источников.....	31
Лабораторная 4. Реализация плагина.	32
Лабораторная 5. Пояснительная записка и защита.....	35
Список используемых источников	49
Приложение А (справочное) Пример ТЗ	51
Приложение Б (обязательное) Пример титульного листа	66
Приложение В (обязательное) Стандарт оформления кода	67
Приложение Г (обязательное) Руководство по настройке инструментов разработки	74

Сокращения

API – Application Programming Interface (программный интерфейс приложения),

MVVM – Model View ViewModel (модель, представление, модель представления),

UML – Unified Modeling Language (унифицированный язык моделирования),

WPF – Windows Presentation Foundation,

АС – автоматизированная система,

ЕСКД – единая система конструкторской документации,

ЕСПД – единая система программной документации,

ОЗУ – оперативное запоминающее устройство,

ООП – объектно-ориентированное программирование,

ОС – операционная система,

ОС ТУСУР – образовательный стандарт томского государственного университета систем управления и радиоэлектроники,

ОЭ – опытная эксплуатация,

ПК – персональный компьютер,

ПО – программное обеспечение,

САПР – система автоматизированного проектирования

ТЗ – техническое задание,

ФИО – фамилия имя отчество,

ЦП – центральный процессор.

Лабораторная 1. Выбор предмета проектирования

Выбор детали и САПР.

К данному этапу нужно отнестись ответственно, т.к. от него зависит то, насколько сложно вам будет выполнить все последующие лабораторные работы. Нужно выбрать проектируемую деталь и определиться, в какой САПР будет она строиться.

Для выбранной детали нужно продумать пять задаваемых пользователем параметров, два из которых зависят друг от друга. Цвет, материал, параметры, которые задаются одной галочкой вкл./выкл., будут считаться как половина параметра и связанными параметрами являться не могут.

Зависимые параметры должны отвечать следующим требованиям: они вводятся через поля для ввода (TextBox), изменение одного должно повлечь изменение допустимого для ввода диапазона значений другого параметра и наоборот.

Пример зависимых параметров.

Рассмотрим пример зависимых параметров. На рисунках 1.1-1.2 являются зависимыми следующие параметры: радиус $R1$ и ширина $L1$, M является константой. Их зависимость выражается формулой $2R1 + M \leq L1$. Данное выражение означает, что цилиндрическая часть детали всегда находится внутри прямоугольной с определенным отступом и не может выходить за ее рамки. Если пользователь введет значение $R1$ или $L1$, которое не будет удовлетворять данному выражению, то должна появиться ошибка валидации.

Например, изначально указаны параметры: $R1 = 10$, $L1 = 50$, $M = 5$. Таким образом: $2 \cdot 10 + 5 \leq 50$, т.е. $25 \leq 50$ – выражение корректно. Предположим, что пользователь изменяет значение $R1$ с 10 на 25, тогда:

$2 \cdot 25 + 5 \leq 50$, т.е. $55 \leq 50$ – выражение некорректно.

В пользовательском интерфейсе необходимо отобразить информацию о том, что какой-либо из данных параметров нужно поменять (заменить

значение на удовлетворяющие условию $R1 \leq (L1 - M)/2$, в данном случае $R1 \leq 22.5$, либо $L1 > 55$).

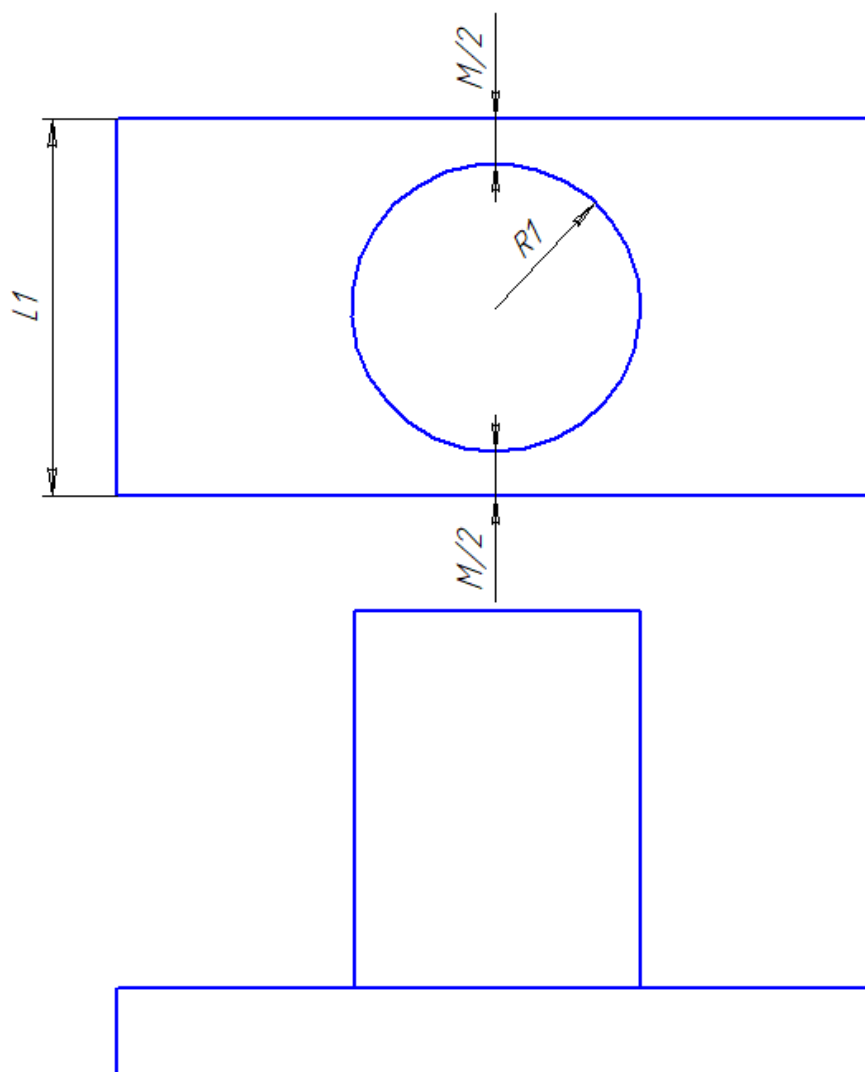


Рисунок 1.1 – Чертеж объекта моделирования

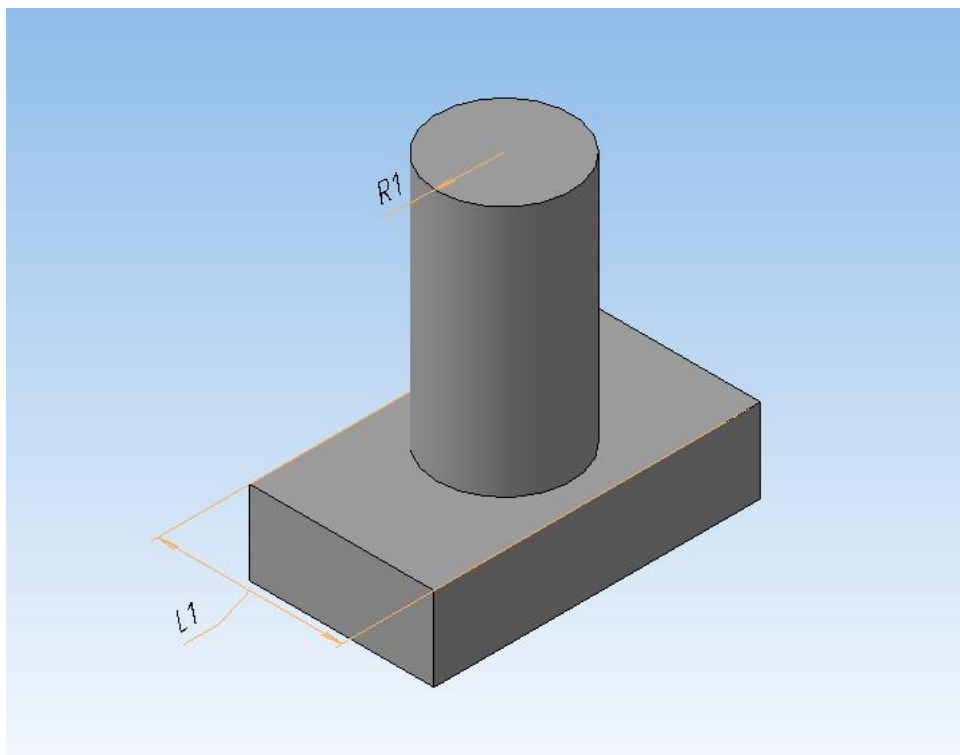


Рисунок 1.2 – Трехмерное представление объекта моделирования

Перед тем, как утвердить выбранную деталь, обязательно нужно построить ее самостоятельно в выбранной САПР. Нужно записать, какие действия были проделаны для построения. Также нужно проверить, можно ли будет повторить их программно через **интерфейс программирования приложения** (Application Programming Interface или API) САПР. Нужно посмотреть в документации, имеются ли там соответствующие методы. Также будет полезно выписать эти методы (это понадобится в следующих лабораторных работах). Подумайте, как можно было бы упростить эту последовательность действий, чтобы она могла быть проще реализована в виде программного кода. Помните, что довольно простые операции в пользовательском интерфейсе могут быть сложно реализуемы через API и наоборот.

Уже на моменте выбора детали нужно примерно понимать, какие повторяющиеся действия стоит вынести в методы, какие потребуются программные сущности.

Создание репозитория.

Также в этой же лабораторной работе нужно создать репозиторий, в котором будет храниться документация и исходный код. При создании репозитория необходимо помнить следующее:

- добавить `.gitignore` для выбранного языка программирования. При создании репозитория файл `.gitignore` можно выбрать при создании репозитория на сайте github.com[1] или добавить его уже после создания репозитория. Актуальные версии `.gitignore` файлов можно найти в [2];

- в процессе разработки можно создавать такое количество веток, которое необходимо для разработки. Можно работать по `gitflow` [3] или по другой модели ветвления `git`, но обязательно у вас должна быть ветка с актуальным вариантом документации и стабильной версией программы. В этой ветке не должно быть закомментированного кода или известных вам ошибок, там должна находиться полностью готовая для просмотра и использования другими людьми функциональность и документация.

В результате выполнения лабораторной работы №1 необходимо:

- предоставить выбранную деталь для построения и САПР. Указать название детали и САПР и предоставить изображение детали. Название плагина сделать в формате «Разработка плагина <Название_детали/Название_объекта> для САПР <Название_САПР>»;

- создать репозиторий на сайте github.com[1] и предоставить ссылку на него.

Лабораторная 2. Техническое задание

Техническое задание (ТЗ) – крайне важный документ в процессе разработки ПО, который фиксирует основные требования и характеристики будущего продукта. Современные условия зачастую требуют адаптировать функциональность программы во время разработки под нужды заказчика. Отсюда повсеместное применение гибких (Agile) методологий разработки ПО [4]. Но как артефакт разработки ТЗ важно тем, что в нем отражены изначальные требования к программе, цель и задачи разработки ПО, а также процессы, которые необходимо автоматизировать и упростить. И даже если спустя несколько итераций разработки изначальные требования станут представлять для заказчика меньший интерес, чем несколько новых появившихся в процессе, у разработчика будет на руках документ, в котором явно отражена согласованная всеми сторонами концепция и назначение разрабатываемого продукта.

Например, если изначально в ТЗ назначение некоторой программы было обозначено как «Плагин «Проектирование болтов» для САПР Kompas 3D», а спустя пару месяцев заказчик хочет функциональность, которая требует разработки модулей, позволяющих проектировать детали любой сложности, рассчитывать сопротивление при изготовлении из разных материалов и, например, делать сборки из нескольких деталей, то у разработчика будет веское основание для увеличения сроков, стоимости или даже пересмотра всего формата взаимодействия с заказчиком.

Диапазоны допустимых значений параметров, описываемых в ТЗ, должны быть приведены в единицах измерения согласно СИ. Представленные диапазоны должны быть адекватны реальным объектам, для которых производится проектирование. Также размер **не может быть** равен или меньше нуля. Как пример неадекватно заданного диапазона можно привести диапазон для диаметра ручки киянки от 0 до 1 м — киянка без ручки или с ручкой диаметром в метр физического смысла не имеет.

При написании ТЗ для выбранного предмета нужно использовать ГОСТ 34.602-2020 [5] для написания разделов. **Оформление текста необходимо взять из ОС ТУСУР [6].** В четвертом разделе ГОСТ 34.602-2020 описаны основные разделы, которые должно содержать ТЗ. Некоторые части ТЗ являются не применимыми в данной лабораторной работе, поэтому ниже приведен список обязательных разделов, которые нужно описать в ТЗ:

1. общие сведения;
 - 1.1. полное наименование автоматизированной системы (АС) и ее условное обозначение;
 - 1.2. наименование организации — заказчика АС, наименование организации-разработчика (при наличии сведений о ней);
 - 1.3. перечень документов, на основании которых создается АС, кем и когда утверждены эти документы;
 - 1.4. плановые сроки начала и окончания работ по созданию АС;
2. цели и назначение создания автоматизированной системы;
 - 2.1. цели создания АС;
 - 2.2. назначение АС;
3. требования к автоматизированной системе;
 - 3.1. требования к структуре АС в целом;
 - 3.2. требования к функциям (задачам), выполняемым АС;
 - 3.3. требования к видам обеспечения АС;
 - 3.4. общие технические требования к АС.
4. состав и содержание работ по созданию автоматизированной системы;
5. порядок разработки автоматизированной системы;
 - 5.1. порядок организации разработки АС;
 - 5.2. перечень документов и исходных данных для разработки АС;
 - 5.3. перечень документов, предъявляемых по окончании соответствующих этапов работ;
6. порядок контроля и приемки автоматизированной системы;
 - 6.1. виды, состав и методы испытаний АС и ее составных частей;

6.2. общие требования к приемке работ, порядок согласования и утверждения приемочной документации;

7. требования к документированию;

7.1. перечень подлежащих разработке документов;

7.2. вид представления и количество документов;

7.3. требования по использованию ЕСКД и ЕСПД при разработке документов;

8. источники разработки.

8.1. документы и информационные материалы (технико-экономическое обоснование, отчеты о законченных научно-исследовательских работах, информационные материалы на отечественные, зарубежные системы-аналоги и др.), на основании которых разрабатывалось ТЗ и которые должны быть использованы при создании АС.

В приложении А приведён пример ТЗ.

Лабораторная 3. Проект системы

Для сдачи проекта системы нужно проработать следующие разделы:

1. Титульный лист;
2. Описание САПР:
 - 2.1. Описание программы (Информация о выбранной САПР ~1 страница);
 - 2.2. Описание API;
 - 2.3. Обзор аналогов плагина;
3. Описание предмета проектирования (берется из ТЗ);
4. Проект системы:
 - 4.1. UML диаграмма классов;
 - 4.2. Макеты пользовательского интерфейса;
5. Список источников.

В качестве стандарта оформления нужно использовать ОС ТУСУР [4].

1 Титульный лист

За основу стандарта оформления взять титульный лист отчетов по лабораторным работам. Титульный лист должен иметь следующие элементы:

- полное наименование высшего учебного заведения;
- наименование кафедры (полное название и сокращение в скобках), на которой сдается лабораторная работа;
- название выполняемой работы;
- название документа;
- название дисциплины;
- группа, ФИО, слева от которой должна быть место под подпись, сдающего студента, а также дата составления проекта системы;
- ученая степень, должность, ФИО, слева от которой должна быть место

под подпись принимающего преподавателя, поле для ввода даты проверки документа;

– место и год в нижнем колонтитуле.

Пример представлен в приложении Б.

2 Описание САПР

2.1 Информация о выбранной САПР

Дать краткое определение САПР, рассказать об основных возможностях выбранной САПР. Найти и добавить аналоги этой САПР. Если не удалось найти прямые аналоги САПР, то привести косвенные. Написать, почему выбрана именно эта САПР.

Пример аналога САПР Kompas-3D[7] является Autodesk Inventor[8].

Пример косвенного аналога для САПР интегральных СВЧ схем Cadence AWR Microwave Office Software[9] (если откинуть прямые аналоги) является программа аналогового и цифрового моделирования электрических и электронных цепей Micro-Cap[10].

2.2 Описание API

Дать определение API, показать структуру API выбранной САПР и дать краткое руководство подключения и работы с ней. Далее описать основные классы в виде таблицы, которые будут использоваться из API. Формат таблицы классов представлен ниже:

– **свойства, константы и поля:** название элемента, тип значения, краткое описание (что хранит в себе данный элемент). Пример представлен в таблице 3.1;

– **методы:** название метода, принимаемые параметры (если нет параметров ставить символ длинное тире «—»), возвращаемое значение (или void, если ничего не возвращается) и краткое описание (что делает данный метод). Пример представлен в таблице 3.2.

Таблица 3.1 – Используемые свойства класса Application

Название	Тип данных	Описание
Documents	Documents	Свойство, содержащие все открытые документы в Inventor
FileManager	FileManager	Свойство, позволяющее работать с файлами Inventor
TransientGeometry	TransientGeometry	Свойство, содержащие методы геометрии

Таблица 3.2 – Используемые метода класса Application

Название	Входные параметры	Тип возвращаемых данных	Описание
CreatePoint2d	int X, int Y	Point2d	Создает точку на 2D эскизе

Сведения о всех классах API и их описание можно найти в сопроводительной документации к САПР. Не нужно брать случайные классы. Нужно выбрать только те элементы API, которые, предположительно, понадобятся для реализации плагина. Для того чтобы понять, какие понадобятся классы и методы, можно реализовать самостоятельно выбранную модель и записать, какие действия были совершены, т.к. названия классов, полей и методов, зачастую, имеют такое же название, как и используемые инструменты.

2.3 Обзор аналогов плагина

Внимание! Не путайте данный пункт с аналогом САПР (2.1).

В данном пункте нужно найти прямые или косвенные (если не удалось найти прямые) аналоги. **Минимальное количество аналогов – 2.** В описании

аналога нужно добавить:

- название аналога с **указанием источника** (официальный сайт или ссылка на страницу, с ее описанием);
- основная функциональность (описать, что сходится и различается с вашим плагином);
- скриншот пользовательского интерфейса.

Если аналог является косвенным, то написать – в чем заключается схожесть приложения с реализуемым плагином.

Пример косвенных аналогов

Пример №1. Косвенный аналог для *плагина создания ограждения (заборов)* является программа для 3D дизайна и архитектурного проектирования SketchUp, т.к. с помощью данного приложения можно создавать не только интерьер дома, но и его внешнюю часть (экстерьер).

Пример №2. Другим примером косвенного аналога является плагин для создания моделей пресс-форм по отношению к плагину создания моделей ящика для деталей. И то, и другое можно представить как матрицу повторяющихся углублений на прямоугольной форме – у пресс-формы (рисунок 3.1) углубления в виде объекта, для создания которого она предназначена, у ящика для деталей (рисунок 3.2) – углубления, полученные выдавливанием прямоугольных эскизов.



Рисунок 3.1 – Фотография пресс-формы[11]

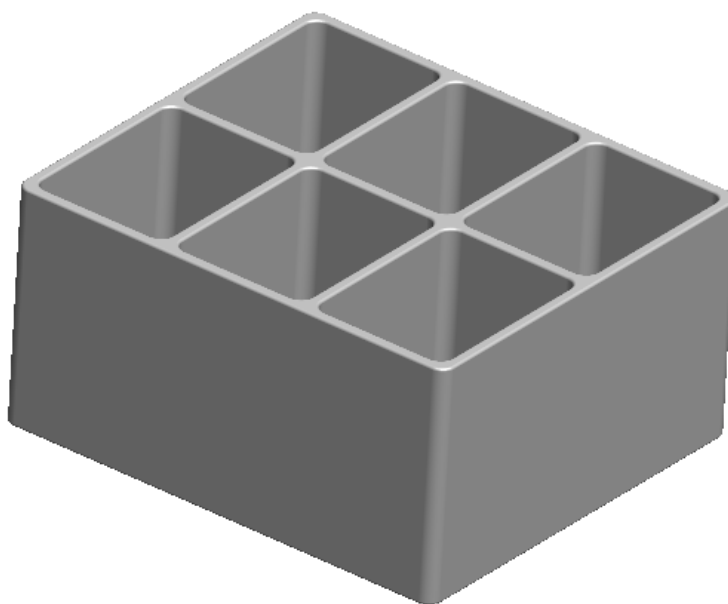


Рисунок 3.2 – Модель ящика для деталей[12]

3 Описание предмета проектирования

В данном разделе нужно дать определение проектируемой модели со **ссылкой на источник**. После определения нужно добавить чертеж и описание параметров из технического задания, которое было сделано в лабораторной работе №2.

4 Проект системы

4.1 Диаграмма классов

В данном разделе нужно дать определение UML диаграмме классов (**со ссылками на источники**). Для создания диаграммы классов рекомендуется использовать приложение Enterprise Architect[13] любой версии. Для составления UML диаграммы классов могут понадобиться следующие виды связи: [14]

– **Агрегация** – это связь двух классов, где первый класс (агрегирующий класс) содержит объект второго класса (агрегируемый класс). При данной связи время жизни двух объектов является разной, т.е. они могут существовать независимо друг от друга, даже если один из них будет уничтожен. Пример связи показан на рисунке 3.3. Символ ромба указывает на агрегирующий класс.

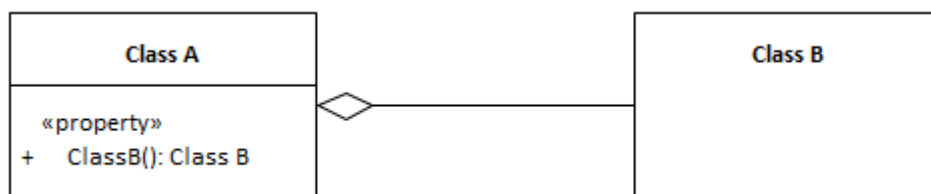


Рисунок 3.3 – Связь агрегация

– **Композиция** – также является связью двух классов, где первый класс (композирующий класс), содержит объект второго класса (композируемый класс). Но в отличие от предыдущей связи, жизнь двух объектов будет

одинаковой, т.е. при уничтожении объекта класса А, объект класса Б уничтожится вместе с ним. Пример связи показан на рисунке 3.4. Символ закрашенного ромба указывает на композирующий класс.

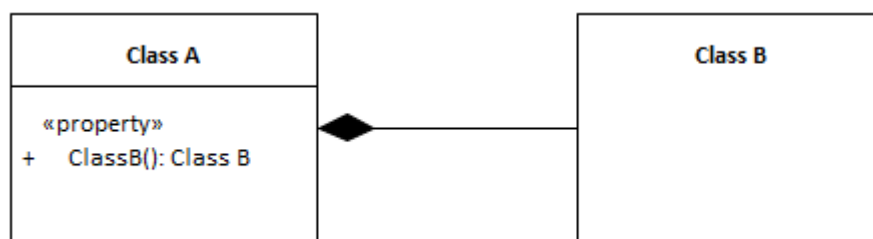


Рисунок 3.4 – Связь композиция

– **Использование.** Данная связь устанавливается тогда, когда один класс использует поля, методы объекта второго класса. Также связь использование устанавливается, когда объект класса используется целиком, но жизнь такого объекта будет ограничена в рамках одного метода. Пример связи представлен на рисунке 3.5. Стрелка указывает на объект, который используется (в данном случае класс А использует класс В).

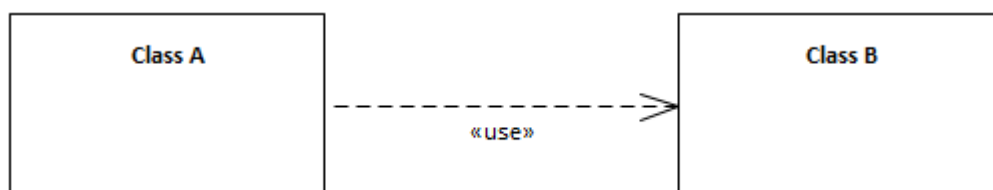


Рисунок 3.5 – Связь использование

– Связь **реализация** используется в том случае, когда существует элемент, который определяет какое-либо поведение (в данном случае это интерфейс), а второй элемент обязательно должен реализовать данное поведение. Данная связь применяется, когда класс реализует интерфейс. Пример такой связи представлен на рисунке 3.6. Стрелка указывает на определяющий поведение элемент (интерфейс).

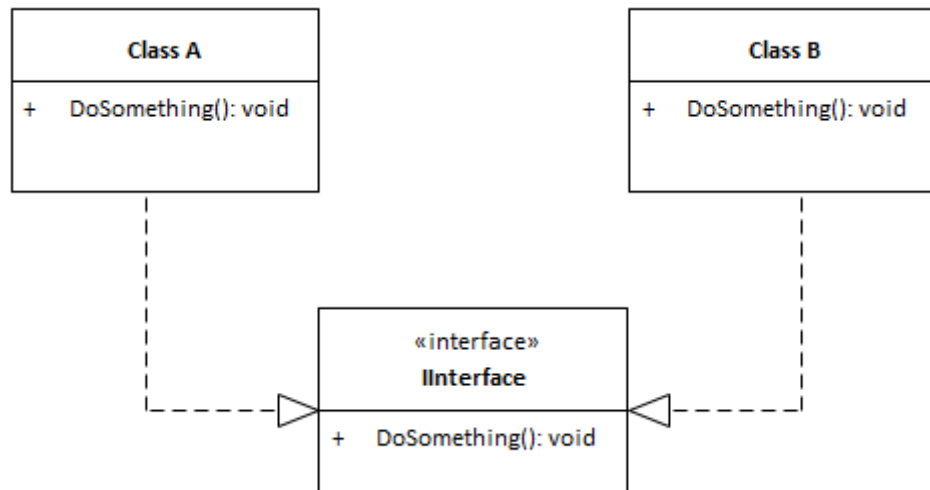


Рисунок 3.6 – Связь реализация

– Связь **наследование** используется, если специализированный элемент (потомок) строится по спецификациям обобщенного элемента (родителя). Пример данной связи представлен на рисунке 3.7. Стрелки указывают на элемент родителя.

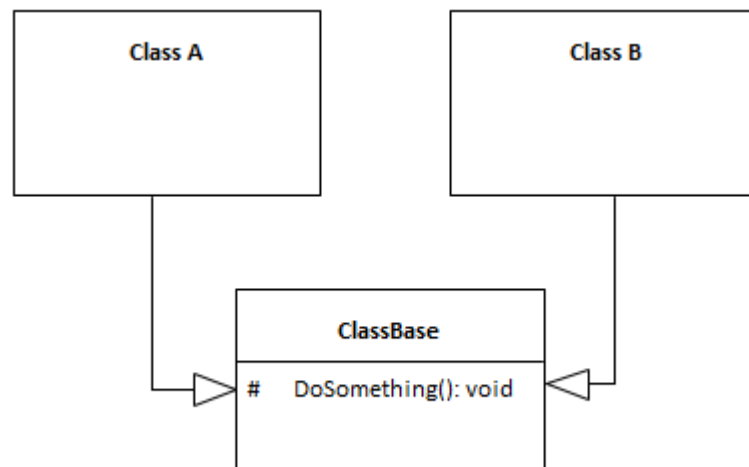


Рисунок 3.7 – Связь реализация

Также можно ознакомиться с источниками [14-16] для чуть более глубокого понимания UML-диаграмм классов.

Пример описания диаграммы классов

Разберем данные связи на примере диаграммы классов, которая показана на рисунке 3.8.

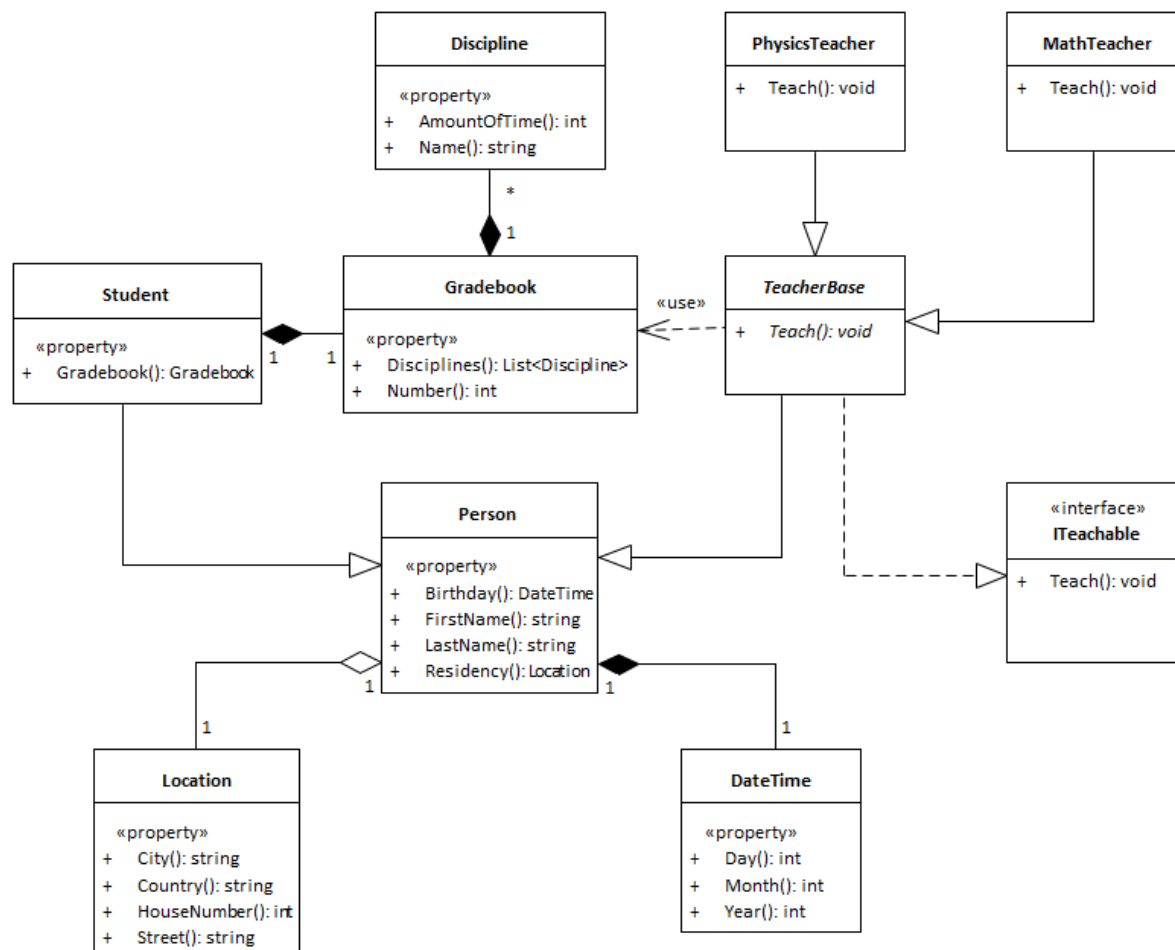


Рисунок 3.8 – Пример UML-диаграммы классов

Класс человека (**Person**) содержит в себе два объекта (за исключением имени **FirstName** и фамилии **LastName**): место проживания (**Residency**) типа **Location** и дата рождения (**Birthday**) типа **DateTime**. Связь между классами **Person** и **Location** является агрегацией, т.к. человек может менять свое место жительства, а связь между **Person** и **DateTime** является композицией, т.к. человек не может изменить свою дату рождения.

От класса **Person** идет наследования к двум другим классам: студент (**Student**) и базовый класс преподавателя (**TeacherBase**). **Student** композитует в себе зачетную книжку (**Gradebook**), которая в свою очередь композитует

коллекцию дисциплин (**Discipline**) (это можно понять по связи «один ко многим»).

TeacherBase реализует интерфейс **ITeachable**, но устанавливает метод **Teach** как абстрактный (это можно понять по курсиву в названии метода). Сам класс **TeacherBase** является абстрактным, т.е. его нельзя создать напрямую, поэтому от данного класса наследуются два других класса: преподаватель физики (**PhysicsTeacher**) и математики (**MathTeacher**).

Плагин можно реализовать двумя способами, они отличаются способом запуска плагина и САПР:

Запуск САПР из плагина.

В данном способе сначала запускается плагин. После ввода нужных данных и нажатия кнопки «Построить» открывается САПР и строится нужная модель. Пример диаграммы классов представлен на рисунке 3.9. Для реализации данного варианта существует несколько подходов. Рассмотрим два варианта.

– **Использование одного общего класса для всех параметров (рисунок 3.9).** В данной реализации используется один класс для описания параметров – **Parameters**. Для каждого свойства описываются свои методы валидации, ограничения и сами значения внутри одного класса **Parameters**. Данный вариант является простой реализацией, но имеет существенный недостаток – малая гибкость. Данная проблема связана с тем, что такая сущность, как **Parameters**, берет большое количество обязанностей. Для исключения этого недостатка можно доработать архитектуру, применив принципы объектно-ориентированного программирования (ООП), которые описаны далее.

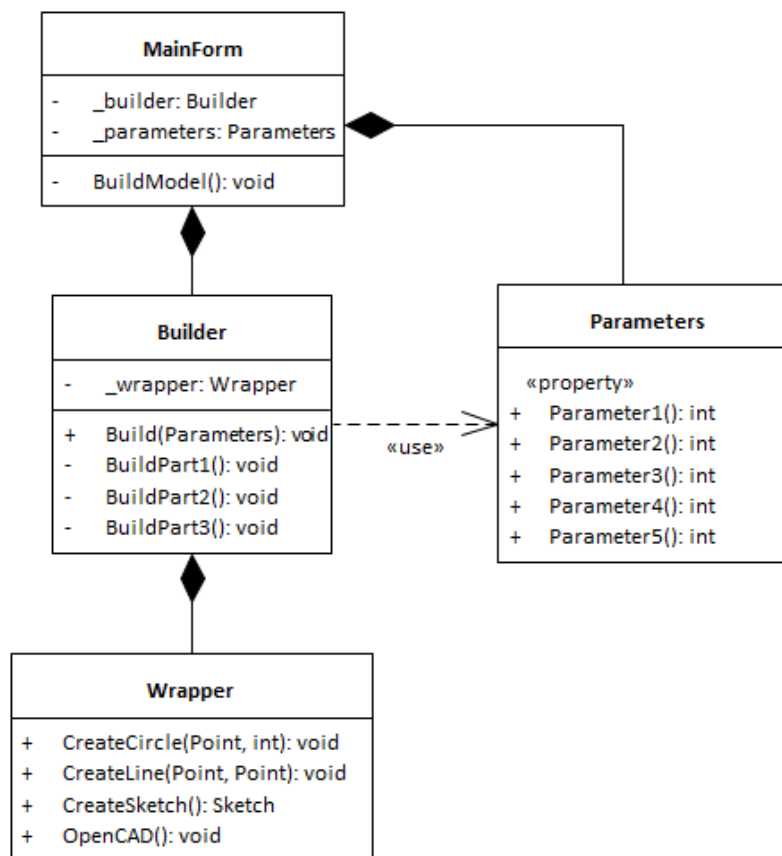


Рисунок 3.9 – Пример первой версии архитектуры плагина как отдельного приложения

– **Использование класса коллекции объектов параметров (рисунок 3.10).** В данном варианте Parameters используется не как хранилище числовых свойств, а как коллекция объектов класса Parameter, внутри которого описаны ограничения и само значение. Для определения, какой объект класса Parameter относится к какому параметру, создается перечисление с обозначением имени каждого параметра. Все параметры хранятся в словаре класса Parameters, где ключом является перечисление, а значением объект класса Parameter.

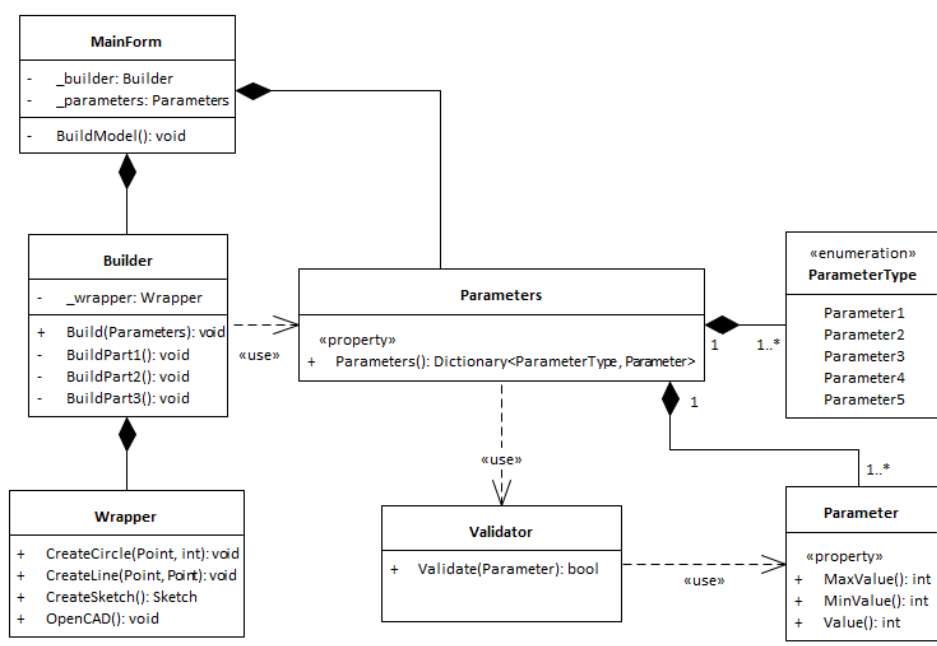
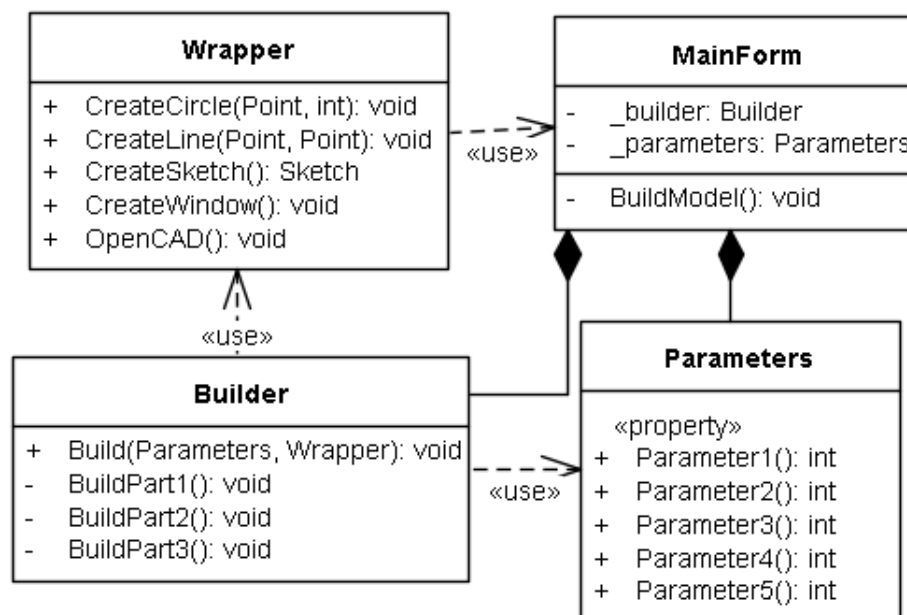


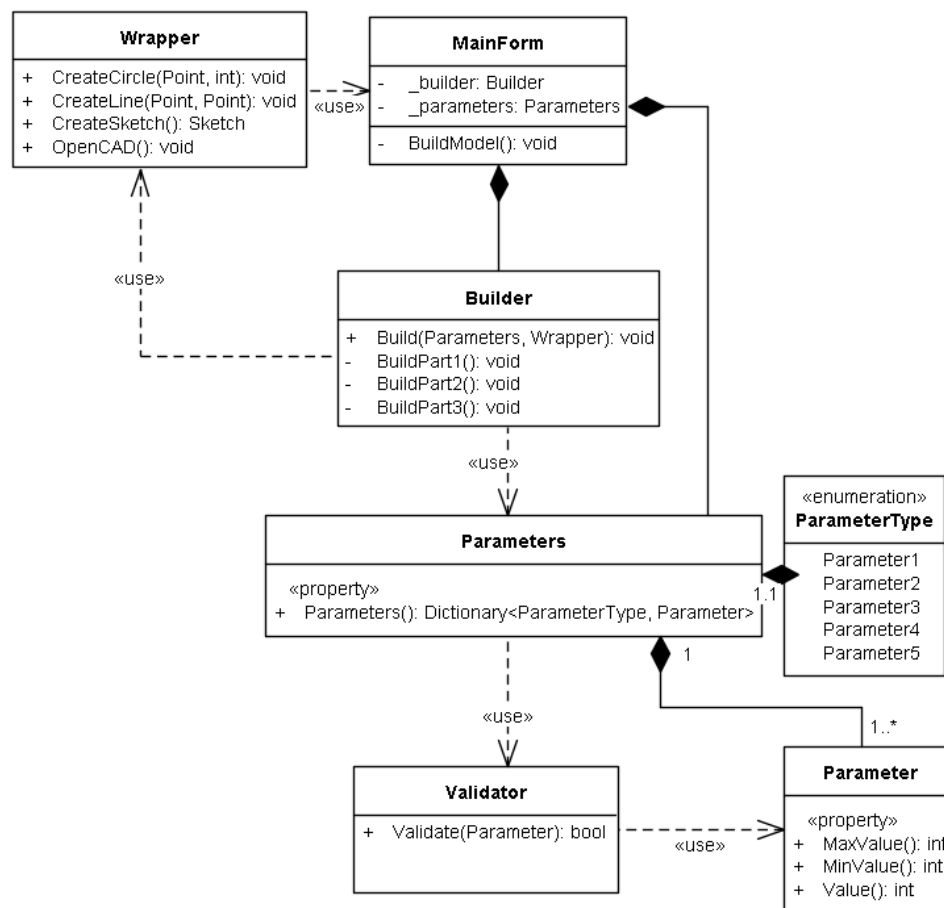
Рисунок 3.10 – Пример второй версии архитектуры плагина как отдельного приложения

Запуск плагина из САПР.

В данном варианте сначала запускается САПР, в которую был добавлен плагин. После запуска открывается плагин через панель инструментов САПР, вводятся значение и строится нужный объект. Пример диаграммы классов представлен на рисунке 3.11. Данный подход также имеет два способа реализации, которые были описаны выше. В данной реализации остается такой же набор классов, что и у первого варианта. Отличиями, не считая разницы в запуске плагина, являются связи между классами, а также сущность Wrapper. Он может быть статическим, благодаря чему можно будет не передавать объект Wrapper в Bulder, но также он может являться создаваемым (тогда нужно будет его передавать через MainForm в Bulder).



а)



б)

а) – использование общего класса;

б) – использование класса коллекции объектов параметров

Рисунок 3.11 – Пример архитектуры плагина, вызываемого из САПР

Разберем основные классы проекта:

- **MainForm** – является главным окном приложения. Хранит в себе параметры (**Parameters**) и объект класса строителя модели (**Builder**);
- **Parameters** – класс, хранящий в себе все параметры модели;
- **Buider** – класс строитель модели;
- **Wrapper** – класс обертка API САПР. В нем находятся все нужные методы создания примитивов и документов, которые пригодятся для построения модели.

Архитектура состоит из следующих проектов:

- **Model** хранит часть моделей бизнес-логики: валидаторы, классы, связанные с параметрами модели. Название данного проекта может быть Model или Core;
- **View** хранит в себе пользовательский интерфейс плагина. Назвать данный проект именем объекта построения, например, CarPlugin (плагин для построения машин);
- **Wrapper** хранит в себе обертку API и класс построения модели. Назвать данный проект Wrapper, а классы: класс обертки – <название_САПР>Wrapper (например, Kompas3DWrapper); класс построения – <название_модели>Builder (например CarBuilder).
- Если планируется реализация плагина, используя технологию WPF, то нужно использовать паттерн MVVM. В таком случае добавится проект **ViewModel**.

На диаграмме класса текст на сущностях должен быть читаемым в Word в масштабе 100 процентов. Если в книжной ориентации не получается разместить диаграмму классов, то можно попробовать разместить ее в альбомной. Нужно **внимательно проверить** правильность выбранных связей, установлена ли кратность связи.

После чего нужно добавить изображение спроектированной архитектуры, под которой нужно добавить словесное описание связей и описание всех элементов класса в виде таблицы. Примеры таблиц представлены на таблицах

3.3 и 3.4. Формат использовать такой же, как и с описанием используемых компонентов API выбранной САПР.

Таблица 3.3 – Свойства класса ChangeableParameters

Название	Тип данных	Описание
PlateDiameter	double	Возвращает и задает значение параметра тарелки
StakeDiameter	double	Возвращает и устанавливает значение диаметра кола
StakeHeight	double	Возвращает и устанавливает значение высоты кола

Таблица 3.4 – Используемые метода класса Builder

Название	Входные параметры	Тип возвращаемых данных	Описание
BuildJuicer	object element	void	Строит модель соковыжималки
TextBoxValidation	—	void	Подсвечивает поля нужным цветом

При непосредственной реализации может произойти так, что архитектура окажется нереализуемой в том виде, в котором вы её проектировали в Проекте системы. Это нормальная практика разработки приложений. Архитектура, в конечном итоге, может обзавестись дополнительными сущностями со своими методами и свойствами, набор методов и их сигнатуры в проектных сущностях также могут быть пересмотрены в процессе непосредственной разработки программного кода. Самое главное – окончательная архитектура (после разработки) должна соответствовать основам ООП с разделенными обязанностями.

4.2 Макеты пользовательского интерфейса

В макетах пользовательского интерфейса нужно схематично (или как можно точнее) показать размещение элементов пользовательского интерфейса, обозначить области: ввода параметров, их описание и диапазоны с единицами измерений.

Примеры пользовательских интерфейсов

Пример макета пользовательского интерфейса представлен на рисунке 3.12. При открытии приложения должны быть указаны значения по умолчанию в поле для ввода.

Название параметров	Поля для ввода значений параметров	Ограничения параметров
Диаметр тарелки	166	166-226 мм
Диаметр кола	70	70-130 мм
Высота кола	60	60-60 мм
Количество зубцов	10	10-12 шт.
Количество отверстий	90	90-100 шт.

Построить

Кнопка при нажатие на которую будет строится модель в САПР

Рисунок 3.12 — Пользовательский интерфейс

Также здесь нужно подписывать единицы измерения и показать, как будет проходить валидация введенных некорректных значений (рисунок 3.13) и сообщения, которые, предположительно могут быть отображены (рисунок 3.14). **Валидация** – это процесс проверки данных, введенных пользователем,

на соответствие заданным критериям (например, на отсутствие ошибок в данных)[17].

Параметр	Введенное значение	Диапазон
Диаметр тарелки	150	156-226 мм
Диаметр кола	52	70-130 мм
Высота кола	150	60-120 мм
Количество зубцов	5	10-12 шт.
Количество отверстий	9	90-100 шт.

Построить

Рисунок 3.13 — Интерфейс с неправильно введенными значениями параметров

Фигура не может быть построена

ОК

Рисунок 3.14 — Интерфейс попытки построения фигуры с неправильно введенными параметрами

Также существуют другие виды компоновки пользовательского интерфейса. На рисунках 3.15-3.17 показаны другие виды компоновки пользовательского интерфейса.

На рисунке 3.15 показан макет интерфейса, который вместо текстового поля использует числовое поле для ввода. Также есть область с построенным объектом, на котором обозначены все изменяемые параметры. В качестве недостатка можно отметить то, что изменяемые параметры на области никак

не связаны с названиями параметров на форме. Это можно исправить, добавив буквенные обозначения параметров в текстовые описания параметров.

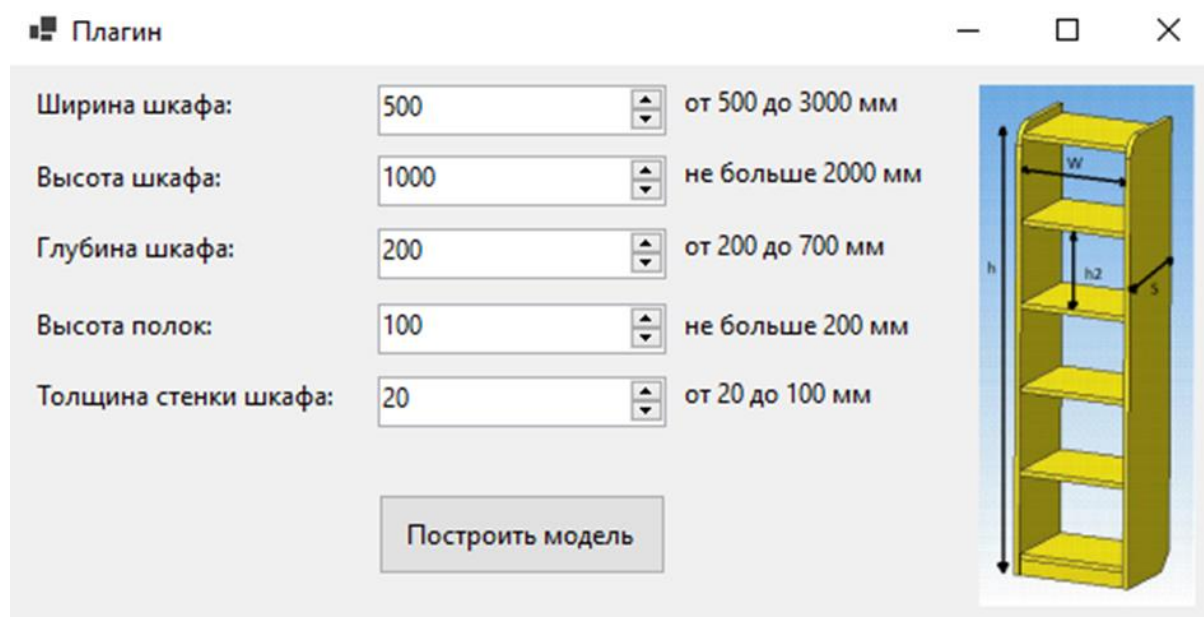


Рисунок 3.15 – Первый пример компоновки пользовательского интерфейса

Особенностью для макета 3.16 является то, что все ошибки сгруппированы в строке состояния с пояснением того, в чем заключается ошибка.

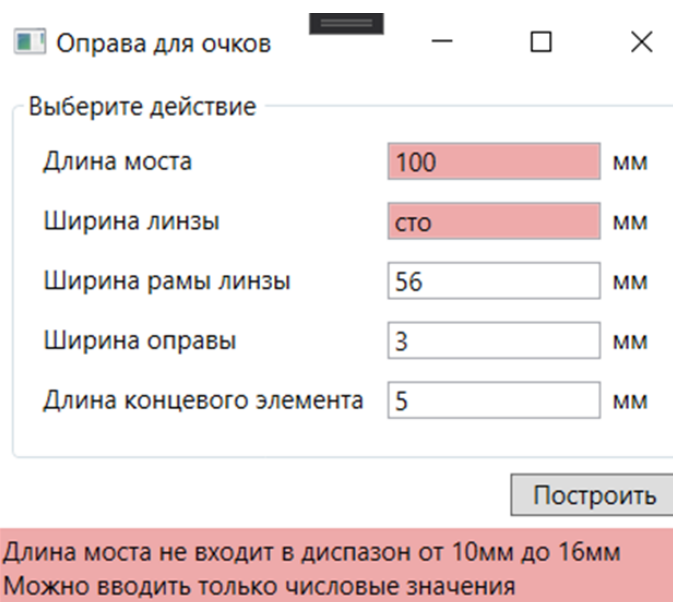


Рисунок 3.16 – Второй пример компоновки пользовательского интерфейса

В макете 3.17 изображена модель построения с возможными параметрами изменения. В описание параметров добавлено обозначение с изображения и исправлены недостатки с макета 3.17.

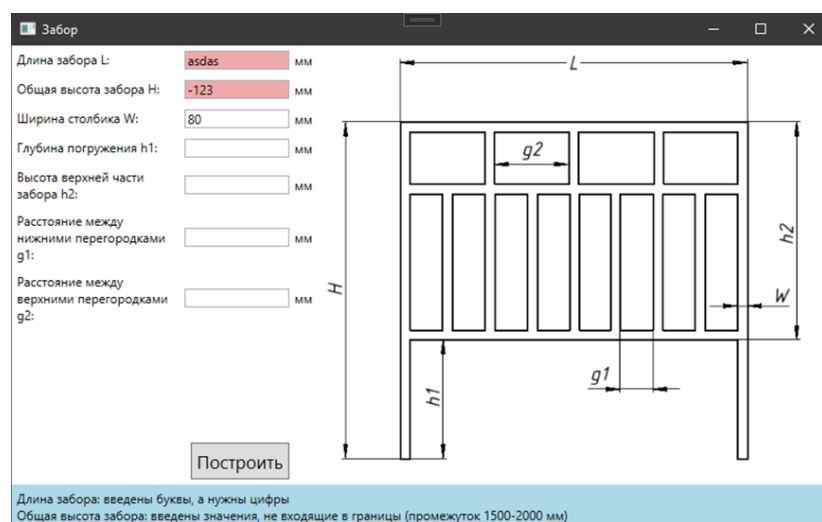


Рисунок 3.17 – Третий пример компоновки пользовательского интерфейса

5 Список используемых источников

В списке используемых источников должны находиться источники (электронные или бумажные) в порядке обращения в основном тексте. В качестве источников рекомендуется ссылаться на различные книги и статьи. Также разрешается использовать авторитетные электронные источники.

Лабораторная 4. Реализация плагина

В результате выполнения данной лабораторной работы необходимо представить рабочую версию плагина, функциональность которой отвечает той, что была утверждена в ТЗ. При реализации плагина необходимо руководствоваться архитектурой, разработанной в лабораторной №3. Бизнес-логика приложения должна быть покрыта модульными тестами на 100%. Под бизнес-логикой понимаются следующие классы: классы параметров, классы проверок значений (валидаторы) и др. вспомогательные классы, взаимодействующие с классом параметров, изменяя их значения. Такие классы, в основном, находятся в одном и том же проекте, например, в проекте Core. Проверить покрытие тестами можно с помощью бесплатного плагина Fine Code Coverage[18] для Visual Studio или его аналогами. В процессе разработки архитектура может изменяться, т.к. не были предусмотрены все особенности предметной области. Разработка плагина будет продолжаться и в следующей лабораторной работе.

Также нужно будет придумать минимум три дополнительные функции, которые не были предложены в ТЗ, для внедрения одной или нескольких из них (в зависимости от сложности реализации) по выбору преподавателя в следующей лабораторной работе. Такая ситуация моделирует изменение требований к функциональности разрабатываемого приложения непосредственно перед сдачей приложения заказчику. В промышленной разработке это достаточно частое явление и к нему нужно быть готовым и понимать — как такие незапланированные изменения отразятся на архитектуре разрабатываемого плагина.

Пример возможных дополнительных функциональностей

Пример 1. Рассмотрим пример возможных дополнительных функциональностей для плагина построения модели «Забор» (см. лабораторную работу №2 «Техническое задание» в примерах интерфейсов).

Изначальные изменяемыми параметрами модели являются: длина забора, общая высота забора, глубина погружения столба, высота верхней части забора, ширина столбика, расстояние между нижними перегородками, расстояние между верхними перегородками. Ниже приведены варианты дополнительной функциональности:

1. Построение внутренней части забора как по расстоянию между прутьями, так и по количеству прутьев (от 1 до 20).

2. Изменить валидацию — для каждого поля ввода установить свой ToolTip с ошибкой.

3. Добавить новый API Kompas 3D для построения забора в другой САПР

Пример 2. Рассмотрим пример возможных дополнительных функциональностей для плагина построения модели «Письменный стол». Изначальные изменяемыми параметрами модели являются: длина столешницы, ширина столешницы, высота столешницы, тип ножек, высота ножек, диаметр основания ножки, длина основания ножки. Ниже приведены варианты дополнительной функциональности:

1. Добавить возможность построить полку под столешницей (например, при отметке галочкой в главном окне плагина). Ширина полки должна находиться в диапазоне от $B / 4$ до 200 мм (где B – ширина столешницы), а расстояние от полки до нижнего края столешницы – в диапазоне от $D_{\min} (A_{\min}) * 2$ до $D_{\max} (A_{\max}) * 3$ мм (где D – диаметр основания ножек стола, A – длина стороны основания ножек стола).

2. Добавить возможность заменить ножки стола сплошной боковой панелью с теми же размерами, что и ножки. При этом если изначально был выбран диаметр ножек D , то будет построена закругленная боковая панель с радиусом скругления, равным $D / 2$, а если была выбрана длина основания A , то будет построена боковая панель с длиной основания, равной A .

Во время написания приложения важно использовать автоматизированные инструменты для форматирования кода. Это нужно для

того, чтобы не тратить большое количество времени для оформления кода. Самое главное при использовании не забывать самостоятельно проверять оформление кода, т.к. у каждого разработчика есть свой стиль написания, и он не должен противоречить оформлению кода команды, в которой вы работаете.

Код должен соответствовать стандарту оформления кода, который представлен в **приложении В**. Для строгого соблюдения стандарта нужно установить на каждый проект линтеры, используя [19] с руководством использования линтеров. В Rider нужно установить StyleCop.Analizers и StyleCop.Analizers.Unstable. Они объединятся в один с названием StyleCop.Analizers и по умолчанию ставится последняя версия (unstable). Нужно вручную выбирать версию более новую стабильную версию и устанавливать её.

Для исключения грамматических ошибок в названиях классов, методов, полей и текста в комментариях, нужно установить Spell Checker[20] для Visual Studio. Также в **приложении Г** описано руководство по установке библиотек с русскими и английскими словами.

Бизнес логика (проект Model или Core) должна быть полностью покрыта модульными тестами. Писать код рекомендуется на языке C#. Другие языки также можно использовать, но при возникновении ошибок их исправлять придется самостоятельно, без помощи преподавателя. Для реализации пользовательского интерфейса рекомендуется использовать WinForm или WPF. Если приложение выполнено на платформе WPF, оно должно быть написано согласно паттерну MVVM (Model, View, ViewModel), для этого можно использовать библиотеки Galasoft MVVMLight (прекращена поддержка), либо CommunityToolkit.Mvvm[21].

Для проверки работоспособности программы необходимо иметь с собой ноутбук с готовой сборкой плагина и установленной САПР. Если ноутбука нет, то можно скооперироваться с другими студентами, у которых та же САПР, что и у вас, или воспользоваться виртуальной машиной на кафедральном компьютере.

Лабораторная 5. Пояснительная записка и защита

В лабораторной №5 нужно реализовать дополнительную функциональность, которая была выбрана преподавателем в лабораторной работе №4, и дополнить модульные тесты, если дополнительная функциональность привела к изменению бизнес-логики приложения. Реализация дополнительной функциональности на завершающем этапе разработки покажет гибкость разработанной архитектуры и выступит в виде симуляции ситуации добавления дополнительной функциональности на поздних этапах разработки в реальном проекте. Также по результатам выполнения работы необходимо составить пояснительную записку (ПЗ), которая должна состоять из следующих пунктов:

1. Введение. В данном пункте описываются основные понятия разработки плагина для САПР и вводная часть описания необходимости разработки плагина.

2. Постановка и анализ задачи. В данном пункте нужно описать, какие задачи были определены и их сроки. Описать собственные размышления о задачах: определение возможных “подводных камней” и проблем, какие были использованы ресурсы при анализе, указать положительные и отрицательные результаты анализа, полученные при анализе документации API, вспомогательных источников, в том числе Интернет-источников.

3. Описание предмета проектирования. В данном пункте нужно описать предмет проектирования и показать его в виде изображения (можно взять из проекта системы или технического задания).

4. Выбор инструментов и средств реализации. Описать все инструменты, которые были использованы при создании плагина и документации к нему. Описать технологию, на которой был создан плагин (например, .Net 6 и WinForms), описать использованные библиотеки, инструменты (например, ReSharper, StyleCop и т.д.).

5. Назначение плагина. Описать, какую проблему может решить данный плагин и его назначение (можно взять из проекта системы или технического задания).

6. Обзор аналогов. В данном пункте нужно найти аналог (косвенный или прямой) реализованного плагина. Описать преимущество и недостатки аналогов в сравнении с реализацией (можно взять из проекта системы).

7. Описание реализации. В данном пункте нужно описать UML-диаграммы классов после проектирования и после реализации плагина. Для UML-диаграммы после реализации необходимо привести таблицы с описанием программных сущностей и их членов. Далее необходимо описать различия, акцентируя внимание на главных архитектурных изменениях. Сделать выводы, почему произошли такие изменения.

8. Описание программы для пользователя. В данном пункте нужно показать, как пользователь должен работать с плагином. Описать ошибки, которые обрабатываются плагином, и написать, как можно решить каждую известную проблему.

9. Тестирование плагина. В данном пункте нужно описать три вида тестирования и показать результаты данного тестирования.

9.1. Функциональное тестирование. Показать, как плагин обрабатывает ошибки во время использования плагина, создание моделей на стандартных, минимальных и максимальных значениях параметров, показать результат работы плагина.

9.2. Модульное тестирование. Показать количество написанных модульных тестов, создать таблицу с описанием тестов, показать, что все модульные тесты выполняются успешно. Также необходимо привести скриншот плагина, измеряющего процент покрытия модульными тестами.

9.3. Нагрузочное тестирование. Необходимо запустить построение детали в бесконечном цикле со средними значениями параметров, на каждом шаге цикла необходимо замерить нагрузку ОЗУ и процессора. В пояснительной записке отобразить результаты в виде графиков зависимости изменения

загруженности ОЗУ и процессора от номера итерации построения детали. По полученным данным сделать выводы.

Для проведения нагрузочного тестирования нужно создать новый консольный проект StressTesting (имя может отличаться). Нужно подключить проекты с бизнес-логикой и проект строителя (Builder). Ниже представлен пример реализации нагрузочного тестирования:

```
var builder = new Builder();
var stopWatch = new Stopwatch();
var parameters = new Parameters();
Process currentProcess =
System.Diagnostics.Process.GetCurrentProcess();
var count = 0;
const double gigabyteInByte = 0.000000000931322574615478515625;

while (true)
{
    stopWatch.Start();
    builder.Build(parameters);
    stopWatch.Stop();
    var computerInfo = new ComputerInfo();
    var usedMemory = (computerInfo.TotalPhysicalMemory
        - computerInfo.AvailablePhysicalMemory)
        * gigabyteInByte;
    streamWriter.WriteLine(
        $"{++count}\t{stopWatch.Elapsed:hh\\\:mm\\\:ss}\t{usedMemory}");
    streamWriter.Flush();
    stopWatch.Reset();
}

streamWriter.Close();
streamWriter.Dispose();
Console.WriteLine($"End {new ComputerInfo().TotalPhysicalMemory}");
```

В данном примере создается один Builder и Parameters, в котором изначально устанавливаются стандартные значения. Далее создается файл, в который будут записываться нужные нам данные. После определения начальных данных запускается бесконечный цикл, который строит модель в САПР, считает время построения одной модели и общую занимаемую память. Число 0.000000000931322574615478515625 это обратное число 1073741824, которое показывает количество байтов в одном гигабайте. Это преобразование нужно, чтобы отображать используемую память в операционной системе в гигабайтах, а не в байтах.

В результате данные в файле log.txt будут иметь следующий вид:

1	00:00:02	8.92043264
2	00:00:05	9.005047808
3	00:00:03	9.080287232
4	00:00:12	9.204350976
5	00:00:07	9.264467968
6	00:00:15	9.34017024
7	00:00:04	9.434898432
8	00:00:02	9.483739136
9	00:00:09	9.663721472
10	00:00:10	9.559220224
11	00:00:05	9.840037888
12	00:00:09	9.916198912
13	00:00:04	9.919021056
14	00:00:08	9.899962368
15	00:00:12	9.937539072
16	00:00:08	9.930358784

Столбцы:

1. Номер созданной модели (забора);
2. Время, затраченное на создание модели;
3. Количество занимаемой оперативной памяти от всех процессов в ГБ.

Далее по этим данным строятся графики и делаются выводы. На графиках должны быть подписаны вместе с единицами измерений.

– Для графика «Загрузка ОЗУ» по оси абсцисс описывается номер построения модели (без единицы измерения т.к. показывает номер), по оси ординат потребляемое количество ОЗУ во всей операционной системе, единица измерения гигабайты (ГБ).

– Для описания времени рекомендуется использовать график в виде гистограммы. По оси абсцисс устанавливаются диапазоны времени построения (например, 0–100 мс, 101–200 мс и т.д.) единицы измерения – миллисекунды (мс) или секунды (с) в зависимости от затраченного времени для одной модели. По оси ординат описывается количество построенных моделей, которые входят в промежуток, без единиц измерений.

Устанавливаемая сетка графика должна быть читаемой. Если значения на оси накладываются друг на друга, то нужно увеличить промежуток между значениями. Старайтесь сделать так, чтобы числа на подписей осей делились на числа 1, 5, 10, 50, 100 и т.д.

Нужно оставлять как можно меньше пустого пространства на графике. Например, если потребляемое ОЗУ находится на графике от 6 до 10 ГБ, то не нужно строить график, начало координат которого начинается с 0. В таком случае нужно определить начало координат с 5,5 ГБ, а его конец в 10,5 ГБ, чтобы было видно, что никакие точки не потерялись на данном промежутке.

Если на графике будут отображаться больше одной характеристики, то нужно устанавливать цвета линиям таким образом, чтобы их можно было без сложностей отличить. Не нужно выбирать оттенки одного цвета.

Также, если на графике будут отображаться больше одной характеристики, нужно добавлять легенду для каждой характеристики с лаконичными названиями.

Примеры анализов результатов стресс тестов

Пример 1. На рисунке 5.1 представлен график зависимости памяти ОЗУ от построения модели, а на рисунке 5.2 представлен график зависимости времени от построения модели.

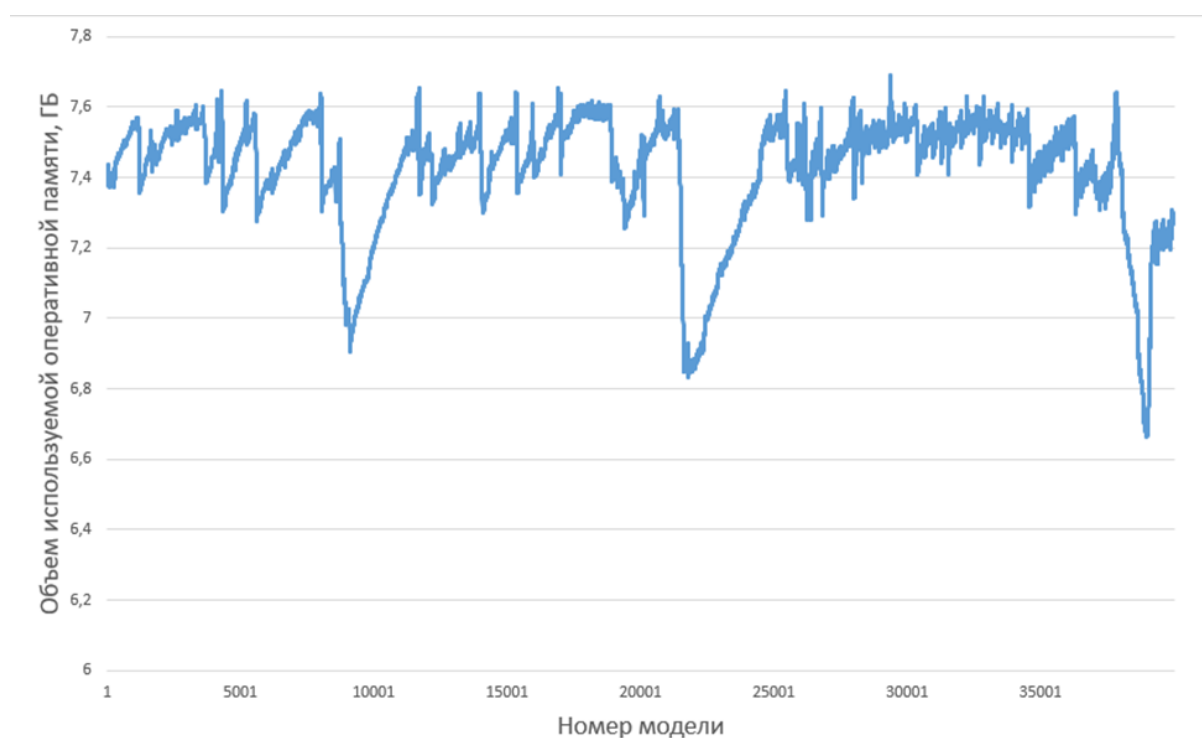


Рисунок 5.1 – График зависимости памяти ОЗУ от количества построенных моделей

Проанализировав график, изображенный на рисунке 5.1, можно сделать вывод, что объем оперативной памяти, затрачиваемый плагином на построение трехмерных моделей, линейно увеличивается до достижения предела объема оперативной памяти. По окончании свободного места оперативная память частично очищается, после чего данный процесс повторяется. Предположительно, это объясняется использованием файла подкачки для компенсации недостатка оперативной памяти. После достижения первого максимума оперативная память уже больше не достигает таких значений и очищается на уровне 7.6 Гб.



Рисунок 5.2 – График гистограммы построения модели

Проанализировав график 5.2 можем сделать вывод, что основное время построение модели от 0 до 200 мс. Это может быть связано с тем, что модель является достаточно простой, из-за чего даже при значительных количествах моделей их нагрузки на оперативную память и процессор недостаточно для замедления построения моделей. Время построения более 200 мс можно связать с загруженностью ОС другими задачами, которые находятся в фоновом режиме.

Пример 2. Нагрузочное тестирование плагина «Письменный стол» проведено на ПК со следующей конфигурацией:

- процессор AMD Ryzen 5 2500U (2 ГГц);
- оперативная память объемом 8 ГБ (доступно 6,9 ГБ);
- видеокарта AMD Radeon Vega 8;
- операционная система Windows 10 Home x64.

В общей сложности проведено три нагрузочных тестирования плагина: со средними, минимальными и максимальными параметрами.

Первое нагрузочное тестирование заключалось в построении трехмерной модели письменного стола с минимальными параметрами:

- длина столешницы L1 – 800 мм;

- ширина столешницы B – 400 мм;
- высота столешницы H1 – 30 мм;
- диаметр основания ножек D – 50 мм;
- высота ножек H2 – 690 мм;
- расстояние между крепежными элементами ручки-скобы – 80 мм;
- количество ящиков для канцелярии N – 3 шт.;
- длина ящиков для канцелярии L2 – 250 мм.

Данное тестирование заняло около 55 минут, по итогам его проведения построено 47842 3D-модели письменного стола. Результаты тестирования приведены на рисунках 5.3 – 5.4.

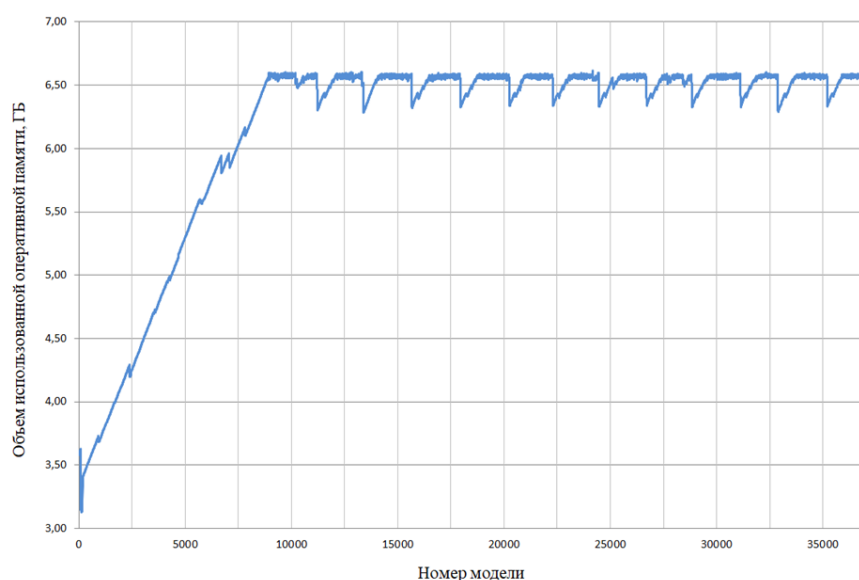


Рисунок 5.3 – График зависимости количества потребляемой оперативной памяти от количества 3D-моделей с минимальными параметрами

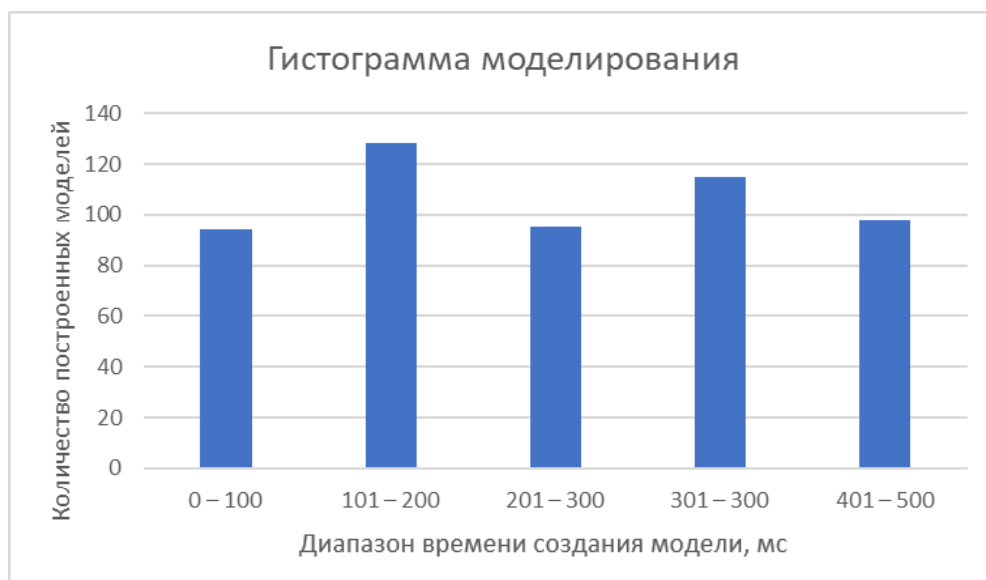


Рисунок 5.4 – График гистограммы построения модели с минимальными параметрами

Второе нагрузочное тестирование заключалось в построении трехмерной модели письменного стола со средними параметрами:

- длина столешницы $L1$ – 1000 мм;
- ширина столешницы B – 625 мм;
- высота столешницы $H1$ – 35 мм;
- диаметр основания ножек D – 60 мм;
- высота ножек $H2$ – 715 мм;
- расстояние между крепежными элементами ручки-скобы – 90 мм;
- количество ящиков для канцелярии N – 4 шт;
- длина ящиков для канцелярии $L2$ – 291 мм.

Данное тестирование заняло около 45 минут, по итогам его проведения построено 37013 3D-моделей письменного стола. Результаты тестирования приведены на рисунках 5.5 – 5.6.

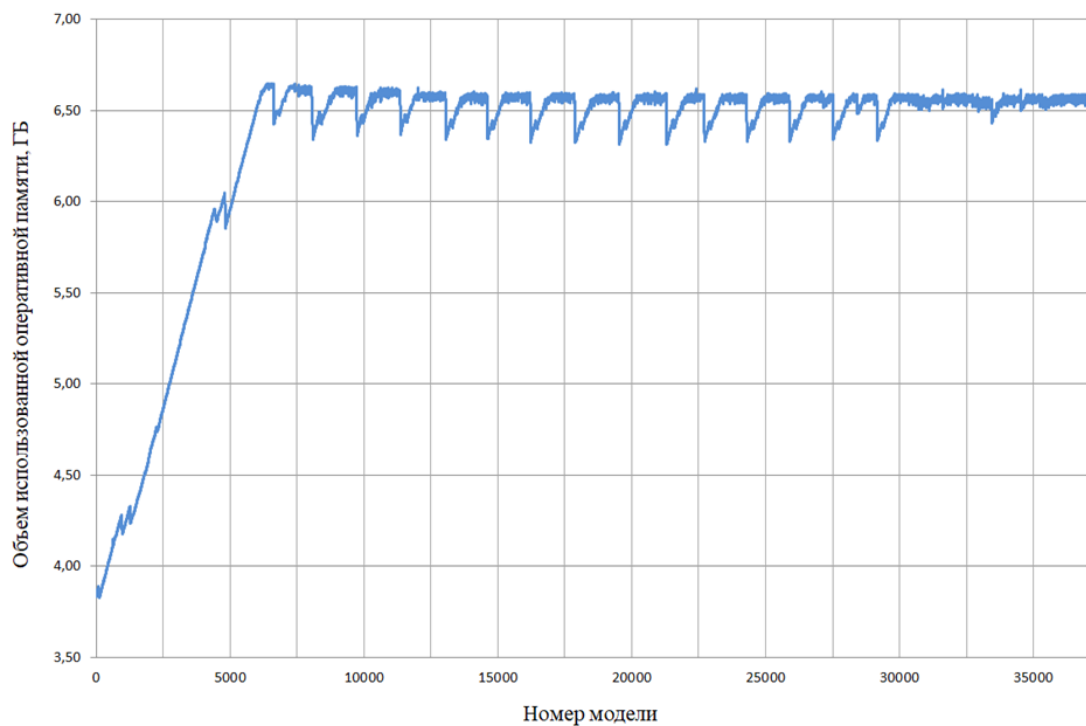


Рисунок 5.5 – График зависимости количества потребляемой оперативной памяти от количества 3D-моделей со средними параметрами

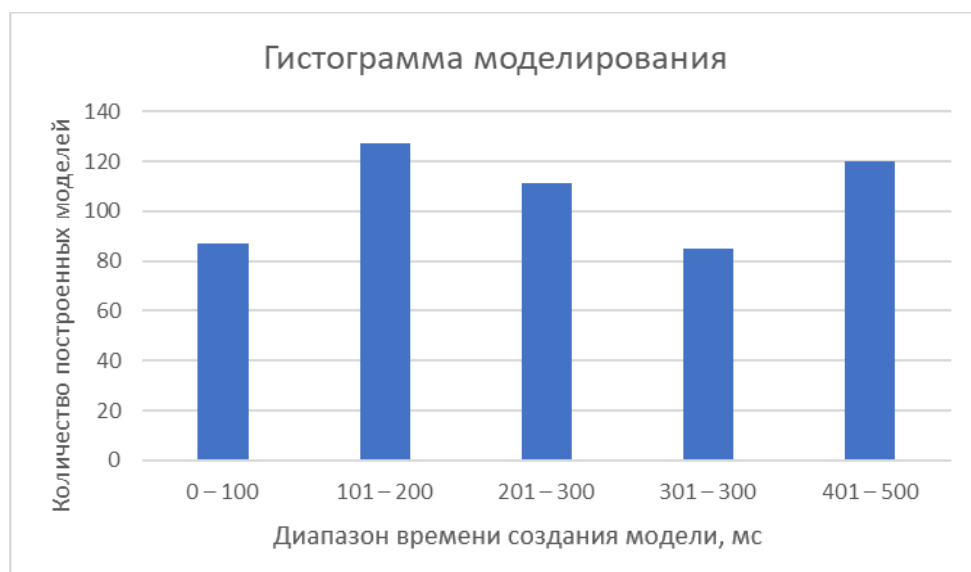


Рисунок 5.6 – График гистограммы построения модели со средними параметрами

Третье нагрузочное тестирование заключалось в построении трехмерной модели письменного стола с максимальными параметрами:

- длина столешницы L1 – 1200 мм;
- ширина столешницы В – 750 мм;
- высота столешницы Н1 – 40 мм;
- диаметр основания ножек D – 70 мм;

- высота ножек Н2 – 740 мм;
- расстояние между крепежными элементами ручки-скобы – 100 мм;
- количество ящиков для канцелярии N – 5 шт;
- длина ящиков для канцелярии L2 – 400 мм.

Данное тестирование заняло около 46 минут, по итогам его проведения построено 30345 3D-моделей письменного стола. Результаты тестирования приведены на рисунках 5.7 – 5.8.

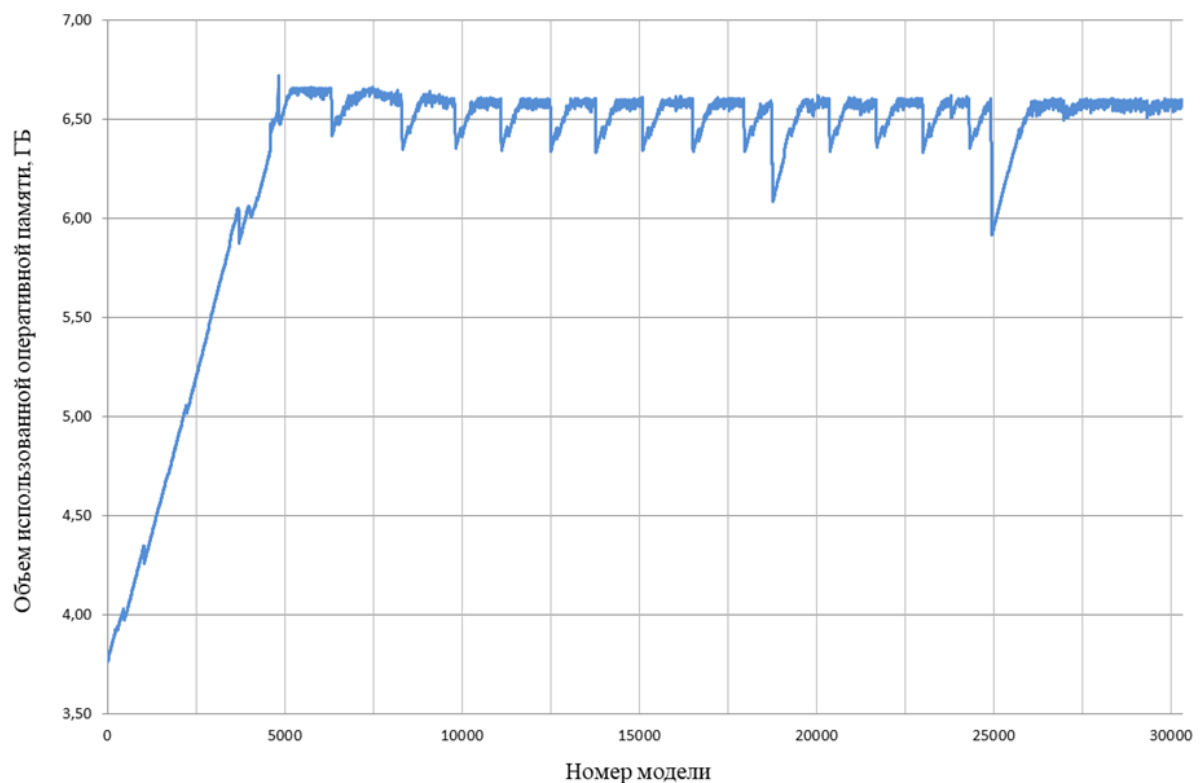


Рисунок 5.7 – График зависимости количества потребляемой оперативной памяти от количества 3D-моделей с максимальными параметрами

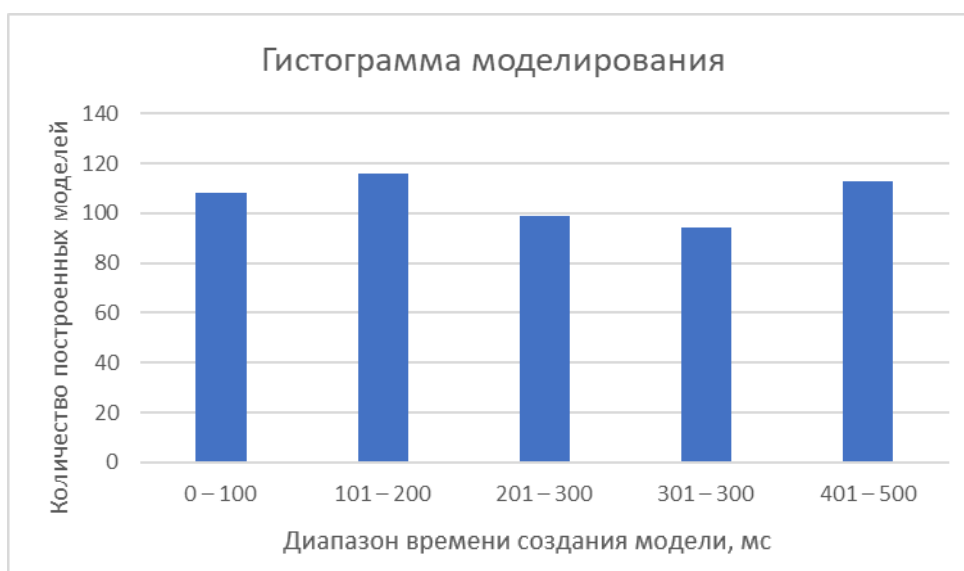


Рисунок 5.8 – График гистограммы построения модели с максимальными параметрами

Исходя из результатов всех трех проведенных тестирований, можно сделать ряд выводов.

Во-первых, объем оперативной памяти, затрачиваемый плагином на построение трехмерных моделей письменного стола, линейно увеличивается до достижения предела объема оперативной памяти. По окончании свободного места оперативная память частично очищается, после чего данный процесс повторяется. Предположительно это объясняется использованием файла подкачки для компенсации недостатка оперативной памяти.

Файл подкачки (виртуальная память, своп-файл) – часть диска, зарезервированная операционной системой для того, чтобы выгружать туда неиспользуемые на данный момент данные и таким образом освободить оперативную память компьютера, объема которой сейчас недостаточно. Иначе говоря, в момент, когда на ПК достигается предел использования оперативной памяти, часть данных из нее перезаписывается в файл подкачки. Вопреки распространенному мнению, это не повышает, а даже наоборот, немного снижает производительность компьютера, однако запись данных в своп-файл позволяет продолжать стабильную работу системы даже в условиях нехватки оперативной памяти.

Во-вторых, во всех трех случаях нагрузочного тестирования наблюдается достижение предела потребления физически доступной оперативной памяти (рисунки 5.3, 5.5, 5.7), однако этот предел достигается тем быстрее, чем выше сложность 3D-модели. Так, объем потребляемой оперативной памяти впервые за соответствующее тестирование составил 6,6 ГБ при построении 9731-й 3D-модели с минимальными параметрами (рисунок 5.3), 6175-й 3D-модели со средними параметрами (рисунок 5.5) и 4809-й 3D-модели с максимальными параметрами (рисунок 5.7). Данную зависимость можно объяснить тем, что параметры 3D-моделей письменного стола являются числами, а чем больше число, тем больше времени занимают любые операции над ним и тем больше требуется для них оперативной памяти компьютера.

В-третьих, как можно заключить по данным графиков на рисунках 5.3, 5.5 и 5.7, при построении 3D-моделей письменного стола с минимальными параметрами используемая оперативная память очищается гораздо реже, чем при построении 3D-моделей со средними или максимальными параметрами. Кроме того, при построении 3D-моделей с минимальными параметрами периодически освобождается объем памяти, меньший по сравнению с тем объемом, что требуется очищать при построении 3D-моделей со средними или максимальными параметрами. Вероятно, это связано с меньшим объемом оперативной памяти, которая требуется для хранения меньших чисел (т.е. минимальных параметров 3D-модели), по сравнению с тем объемом, что необходим для хранения средних или максимальных параметров.

В-четвертых, исходя из графиков, изображенных на рисунках 5.4, 5.6 и 5.8, можно утверждать, что построение не зависит от размеров модели. На всех графиках видно, что большое число моделей строилось за 101–200 миллисекунд.

10. Заключение. Кратко написать о полученных результатах, акцентируя внимание на важные и неожиданные результаты как положительные, так и

отрицательные, сделать выводы. Написать о полученном опыте, который был получен за работу по всем лабораторным работам.

11. Список использованных источников. Все источники, которые были использованы в проекте системы.

Список используемых источников

1. Официальный сайт GitHub [Электронный ресурс]: URL: <https://github.com/> (дата обращения: 15.09.2023).
2. Файлы .gitignore [Электронный ресурс]: URL: <https://github.com/github/gitignore> (дата обращения: 15.09.2023).
3. Рабочий процесс Gitflow Workflow [Электронный ресурс]: URL: <https://www.atlassian.com/ru/git/tutorials/comparing-workflows/gitflow-workflow> (дата обращения: 15.09.2023).
4. Scrum. Революционный метод управления проектами / Джефф Сазерленд ; пер. с англ. М. Гескиной — 2-е изд. — М. : Манн, Иванов и Фербер, 2017. — 272 с.
5. ГОСТ 34.602-2020. Информационные технологии. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы. М.: Стандартинформ, 2020. 13 с.
6. Образовательный стандарт вуза ОС ТУСУР 01-2021. Работы студенческие по направлениям подготовки и специальностям технического профиля. Общие требования и правила оформления. Томск, 2021. 52 с.
7. Официальный сайт Компас 3D [Электронный ресурс]: URL: <https://kompas.ru/> (дата обращения: 28.01.2024).
8. Официальный сайт Autodesk Inventor [Электронный ресурс]: URL: <https://www.autodesk.com/products/inventor/> (дата обращения: 28.01.2024).
9. Официальный сайт Cadence AWR Microwave Office Software [Электронный ресурс]: URL: https://www.cadence.com/en_US/home/tools/system-analysis/rf-microwave-design/awr-microwave-office.html (дата обращения: 28.01.2024).
10. Официальный сайт Micro-Cap [Электронный ресурс]: URL: <https://web.archive.org/web/20230219052113/http://www.spectrum-soft.com/index.shtm#> (дата обращения: 19.02.2023).
11. Сайт компании ПЭТ индустрия [Электронный ресурс]: URL: <https://pet-industrial.ru/izgotovlenie-press-form/> (дата обращения: 28.01.2024).

12. Модель «Ящик для хранения мелких деталей 6 отделений» [Электронный ресурс]: URL: <https://cults3d.com/ru/3d-model/dom/boite-rangements-petites-pieces-6-compartiments> (дата обращения: 28.01.2024).
13. Официальный сайт Enterprise Architect [Электронный ресурс]: URL: <https://sparxsystems.com/> (дата обращения: 28.01.2024).
14. Фаулер М. UML. Основы, 3-е издание. – Пер. с англ. – СПб: Символ-Плюс, 2004. – 192 с.
15. Язык UML. Руководство пользователя. Изд: ДМК Пресс, 2015, с.496
16. Применение UML 2.0 и шаблонов проектирования. Введение в объектно-ориентированный анализ, проектирование и итеративную разработку. Изд: Вильямс, 2013, с.739 (3-е издание).
17. Vice Ruan, Muhammad Shujaat Siddiqi, MVVM Survival Guide for Enterprise Architectures in Silverlight and WPF – Packt Publishing, 2012. – 491 p.
18. Fine Code Coverage [Электронный ресурс]: URL: <https://marketplace.visualstudio.com/items?itemName=FortuneNgwenya.FineCodeCoverage2022> (дата обращения: 24.10.2023).
19. StyleCop Analyzers Implementation in .NET[Электронный ресурс]: URL: <https://code-maze.com/dotnet-stylecop-analyzers-implementation/> (дата обращения: 24.10.2023).
20. Visual Studio Spell Checker [Электронный ресурс]: URL: <https://marketplace.visualstudio.com/items?itemName=EWoodruff.VisualStudioSpellCheckerVS2022andLater> (дата обращения: 28.01.2024).
21. Introduction to the MVVM Toolkit [Электронный ресурс]: URL: <https://learn.microsoft.com/en-us/dotnet/communitytoolkit/mvvm/> (дата обращения: 24.10.2023).
22. Официальный сайт RSDN [Электронный ресурс]: URL: <https://rsdn.org/> (дата обращения: 24.10.2023).

Приложение А
(справочное)
Пример ТЗ

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

**на выполнение в 2023 году работ по разработке плагина "Забор" для
системы автоматизированного проектирования Inventor**

СОДЕРЖАНИЕ

<u>1 ОБЩИЕ СВЕДЕНИЯ</u>	<u>3</u>
<u>1.1 Полное наименование автоматизированной системы</u> и ее условное обозначение	3
<u>1.2 Наименование заказчика и исполнителя</u>	3
<u>1.3 Перечень документов, на основании которых создается АС</u>	3
<u>1.4 Плановые сроки начала и окончания работ по созданию АС</u>	4
<u>2 ЦЕЛИ И НАЗНАЧЕНИЕ СОЗДАНИЯ АВТОМАТИЗИРОВАННОЙ</u> <u>СИСТЕМЫ</u>	<u>5</u>
<u>2.1 Цели создания АС</u>	5
<u>2.2 Назначение АС</u>	5
<u>3 ТРЕБОВАНИЯ К АВТОМАТИЗИРОВАННОЙ СИСТЕМЕ</u>	<u>6</u>
<u>3.1 Требования к структуре АС в целом</u>	6
<u>3.2 Требования к функциям (задачам), выполняемым АС</u>	7
<u>3.3 Требования к видам обеспечения АС</u>	8
<u>3.4 Общие технические требования к АС</u>	9
<u>4 СОСТАВ И СОДЕРЖАНИЕ РАБОТ ПО СОЗДАНИЮ</u> <u>АВТОМАТИЗИРОВАННОЙ СИСТЕМЫ</u>	<u>10</u>
<u>5 ПОРЯДОК РАЗРАБОТКИ АВТОМАТИЗИРОВАННОЙ СИСТЕМЫ</u>	<u>11</u>
<u>5.1 Порядок организации разработки АС</u>	11
<u>5.2 Перечень документов и исходных данных для разработки АС</u>	11
<u>5.3 Перечень документов, предъявляемых по окончании соответствующих</u> этапов работ	11
<u>6 ПОРЯДОК КОНТРОЛЯ И ПРИЕМКИ АВТОМАТИЗИРОВАННОЙ</u> <u>СИСТЕМЫ</u>	<u>13</u>
<u>6.1 Виды, состав и методы испытаний АС и ее составных частей</u>	13
<u>6.2 Общие требования к приёмке работ по стадиям</u>	13
<u>7 ТРЕБОВАНИЯ К ДОКУМЕНТИРОВАНИЮ</u>	<u>15</u>
<u>7.1 Перечень подлежащих разработке документов</u>	15
<u>7.2 Вид представления и количество документов</u>	15
<u>7.3 Требования по использованию ЕСКД и ЕСПД при разработке</u> документов	15
<u>8 ИСТОЧНИКИ РАЗРАБОТКИ</u>	<u>17</u>

1 ОБЩИЕ СВЕДЕНИЯ

1.1 Полное наименование автоматизированной системы и ее условное обозначение

Разработка плагина "Забор" для системы автоматизированного проектирования (САПР) Inventor.

1.2 Наименование заказчика

Заказчиком работ является: кандидат технических наук, доцент кафедры компьютерных систем в управлении и проектировании (КСУП) Калентьев Алексей Анатольевич.

Адрес заказчика: 634045 Томская область Томск ул. Красноармейская 147 СБИ, офис 210.

1.3 Перечень документов, на основании которых создается АС

Выполняемая работа и оформление её результатов должны отвечать требованиям нормативно-правовых актов, а также соответствующих государственных стандартов из числа Комплекса стандартов на автоматизированные системы:

– ГОСТ 34.602-2020 “Информационные технологии. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы”;

– ОС ТУСУР 01-2021 “Образовательный стандарт ВУЗа. Работы студенческие по направлениям подготовки и специальностям технического профиля. Общие требования и правила оформления”;

– ОК 012-93 “Общероссийский классификатор изделий и конструкторских документов (классификатор ЕСКД)”;

– ГОСТ 19.103-77 “Единая система конструкторской документации. Обозначения программ и программных документов”.

1.4 Плановые сроки начала и окончания работ по созданию АС

Плановый срок начала работ: с 23 сентября 2023 года.

Плановый срок окончания работ: не позднее 29 декабря 2023 года.

2 ЦЕЛИ И НАЗНАЧЕНИЕ СОЗДАНИЯ АВТОМАТИЗИРОВАННОЙ СИСТЕМЫ

2.1 Цели создания АС

Целями выполнения работ по разработке плагина "Забор" для САПР Inventor является автоматизация построения заборов.

2.2 Назначение АС

Назначение разрабатываемого плагина обусловлено быстрым моделированием заборов разных типов. Благодаря данному расширению, мастера по заборам могут наглядно рассмотреть спроектированную модель, при необходимости перестроить под необходимые им параметры. На рисунке 2.1 представлена модель забора.

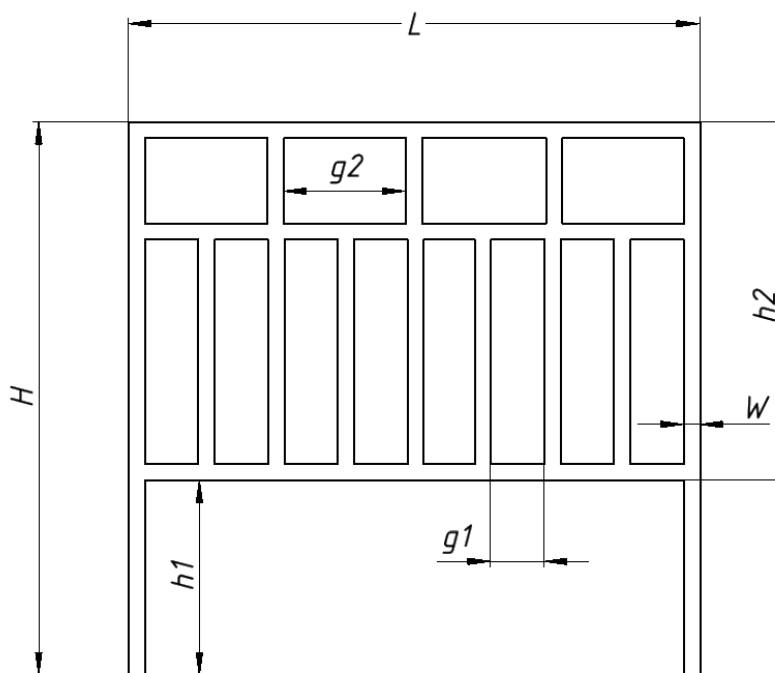


Рисунок 2.1 — Модель забора с размерами

3 ТРЕБОВАНИЯ К АВТОМАТИЗИРОВАННОЙ СИСТЕМЕ

3.1 Требования к структуре АС в целом

3.1.1 Требования к структуре и функционированию системы

Система должна быть выполнена в одном из двух вариантов:

- В качестве встроенного плагина САПР “Autodesk Inventor”, который запускается непосредственно из САПР.

- В качестве сторонней программы, способной запустить процесс программы “Autodesk Inventor” для построения детали.

Изменяемые параметры для плагина (также все обозначения показаны на рис. 2.1):

- длина забора L (1000 — 3000мм);
- общая высота забора H (1500 — 2000мм);
- глубина погружения столба h_1 ($1/3$ — $1/2$ от общей высоты забора);
- высота верхней части забора h_2 ($1/2$ — $2/3$ от общей высоты забора);
- ширина столбика W (10 — 70мм);
- расстояние между нижними перегородками g_1 (не больше длины забора и не меньше ширины одного столбика. Количество столбиков будет определяться автоматически исходя из данного параметра и длины забора);
- расстояние между верхними перегородками g_2 (не больше длины забора и не меньше ширины одного столбика. Количество столбиков будет определяться автоматически исходя из данного параметра и длины забора).

АС должна иметь пользовательский интерфейс с возможностью изменения значений, представленных выше, и последующим построении объекта «Забор» в САПР Inventor. В плагине должны проходить проверки значений, вводимых пользователем. Реализуемый плагин должен обеспечивать обработку ошибочных ситуаций, возникающих в процессе работы. При нажатии на кнопку «Построить» должна проходить проверка

правильности ввода данных. Если данные некорректные, то должно высветиться окно с ошибкой построения и не будут применяться введенные параметры.

3.1.2 Требования к численности и квалификации персонала системы

Дополнительные требования к численности и квалификации персонала системы не предъявляются.

3.1.3 Показатели назначения

Разработанная система должна обеспечивать следующие показатели назначения:

- Время построения детали при учете уже запущенной программы САПР не должно превышать одной минуты;

- Система не должна позволять создавать детали с некорректно заданными параметрами (см. п. 3.1.1 *“Изменяемые параметры для плагина”*).

- Требования к аппаратной части и масштабированию для обеспечения перечисленных показателей должны быть определены на этапе технического проектирования.

3.1.4 Требования к надежности

Дополнительные требования к надежности не предъявляются.

3.1.5 Требования к безопасности

Дополнительные требования к безопасности плагина “Забор” не предъявляются.

3.1.6 Требования к эргономике и технической эстетике

Пользовательские интерфейсы для всех подсистем, разработанных в рамках создания системы должны быть выполнены в виде desktop-интерфейсов с помощью фреймворков WindowsForms, WPF или аналогичных им, позволяющих создавать пользовательские интерфейсы для ОС Windows 10 и выше.

Интерфейсы должны быть адаптированы под минимальную высоту экрана 1080 пикселя и ширину экрана 1920.

Элементы интерфейса должны отвечать рекомендациям по верстке интерфейсов desktop-приложений указанным в источнике [1].

3.1.7 Требования к эксплуатации, техническому обслуживанию, ремонту и хранению компонентов системы

Дополнительные требования к эксплуатации, техническому обслуживанию, ремонту и хранению компонентов системы не предъявляются.

3.1.8 Требования к защите информации от несанкционированного доступа

Дополнительные требования к защите информации от несанкционированного доступа не предъявляются.

3.1.9 Требования по сохранности информации при авариях

Дополнительные требования по сохранности информации при авариях не предъявляются.

3.1.10 Требования к защите от влияния внешних воздействий

Дополнительные требования к защите от влияния внешних воздействий не предъявляются.

3.1.11 Требования к патентной чистоте

Дополнительные требования к патентной чистоте не предъявляются.

3.1.12 Требования по стандартизации и унификации

Разработка системы должна осуществляться в рамках рекомендаций по стандартизации Р 50-54-38-88 “Общесистемное ядро САПР машиностроительного применения. Общие требования”.

3.2 Требования к функциям (задачам), выполняемым АС

3.2.1 Перечень функций, задач или их комплексов

Забор - сооружение, которое ограждает определенный участок местности по периметру или его части. Забор обычно состоит из чередующихся столбов и перекрытий. Поскольку весь забор состоит из повторяющихся участков

достаточно спроектировать одну его секцию и изготовить в нужном количестве.

В рамках задачи должен быть спроектирован и реализован механизм задания параметров с проверкой их корректности, а также разработана система взаимодействия с API САПР “Autodesk Inventor”, производящая построение секции забора по заданным параметрам.

3.3 Требования к видам обеспечения АС

3.3.1 Требования к математическому обеспечению системы

Дополнительные требования к математическому обеспечению системы не предъявляются.

3.3.2 Требования к информационному обеспечению системы

Дополнительные требования по информационному обеспечению системы не предъявляются.

3.3.3 Требования к лингвистическому обеспечению системы

При разработке программы допускается использовать русский и английский языки, при этом не допускается использование обоих одновременно. При реализации сразу двух языков должна быть предусмотрена возможность переключения между ними.

3.3.4 Требования к программному обеспечению системы

При выборе программного обеспечения необходимо отдавать предпочтение платформам разработки и библиотекам, распространяемым под лицензией MIT или аналогичным ей лицензиям, допускающим свободное использование в любом ПО и освобождающим использующих от любой оплаты. Версия САПР Inventor версии 2022.

Помимо этого, разработанная система должна работать на ПК с ОС Windows версии 10 и старше и разрядностью x64 с NET Framework 4.7.2.

3.3.5 Требования к техническому обеспечению системы

ЦП 2.5 ГГц;

16 ГБ ОЗУ;

место на диске — 40 ГБ;

графический процессор с объемом памяти 1 ГБ, пропускной способностью 29 ГБ/с и поддержкой DirectX 11.

3.3.6 Требования к метрологическому обеспечению

Дополнительные требования к метрологическому обеспечению не предъявляются.

3.3.7 Требования к организационному обеспечению

Дополнительные требования к организационному обеспечению не предъявляются

3.4 Общие технические требования к АС

Требования к общим техническим требованиям к АС не предъявляются.

4 СОСТАВ И СОДЕРЖАНИЕ РАБОТ ПО СОЗДАНИЮ АВТОМАТИЗИРОВАННОЙ СИСТЕМЫ

Этапы проведения работ по разработке плагина "Забор" для САПР Inventor приведены в таблице 4.1.

Таблица 4.1 – Этапы проведения работ по разработке плагина "Забор" для САПР Inventor

Этап	Состав работ	Наименование документа	Обозначение	Разработан согласно	Сроки выполнения
1	Создание технического задания	Техническое задание	—	ГОСТ 34.602–2020	Не позднее 30 сентября 2023 года
2	Создание проекта системы	Проект системы	—	ОС ТУСУР 01-2021	Не позднее 15 октября 2023 года
3	Реализация плагина	Программный код	—	RSDN Magazine #1-2004	Не позднее 15 ноября 2023 года
		Документ с тремя вариантами дополнительной функциональности плагина для согласования			
		Модульные тесты			
4	1. Доработка плагина 2. Создание пояснительной записки	Программный код	—	1. RSDN Magazine #1-2004 2. ОС ТУСУР 01-2021	Не позднее 29 декабря 2023 года
		Модульные тесты			
		Пояснительная записка			

5 ПОРЯДОК РАЗРАБОТКИ АВТОМАТИЗИРОВАННОЙ СИСТЕМЫ

5.1 Порядок организации разработки АС

Работа по разработке АС организуется в удаленном формате с возможностью очного присутствия в рабочие часы и использовании для разработки ПК, находящихся в распоряжении кафедры КСУП.

5.2 Перечень документов и исходных данных для разработки АС

Для разработки плагина "Забор" для САПР Inventor нужны следующие документы:

- документация для языка программированию C#;
- ГОСТ Р 52278-2016 «Ограждения защитные. Классификация. Общие положения»;

5.3 Перечень документов, предъявляемых по окончании соответствующих этапов работ

По окончании соответствующих этапов работ должен быть предоставлен следующий перечень документов:

- документ технического задания;
- документ проекта системы;
- программный код;
- пояснительная записка.

6 ПОРЯДОК КОНТРОЛЯ И ПРИЕМКИ АВТОМАТИЗИРОВАННОЙ СИСТЕМЫ

6.1 Виды, состав и методы испытаний АС и ее составных частей

Испытания должны быть организованы и проведены в соответствии с [2-3].

Должны быть проведены следующие виды испытаний:

- предварительные испытания;
- опытная эксплуатация (ОЭ);
- приёмочные испытания.

В предварительные испытания плагина входят следующие пункты:

- модульное тестирование логики;
- нагрузочное тестирование;
- ручное тестирование

В этап опытной эксплуатации входит ручное тестирование.

В этап приемочного испытания входит ручное тестирование.

6.2 Общие требования к приёмке работ по стадиям

Приёмка результатов работ осуществляется поэтапно в соответствии с календарным планом выполнения работ (п. 4).

В процессе приёмки работ должна быть осуществлена проверка системы на соответствие требованиям разработанных ТЗ.

Прочие требования и дефекты системы, выявленные на испытаниях и не относящиеся к требованиям, приведённым в разработанных частных технических заданиях, могут документироваться как желательные

доработки. Наличие желательных доработок не влияет на приёмку работ и процесс передачи системы в эксплуатацию.

Комплектность передаваемой отчётной документации подлежит проверке Заказчиком.

7 ТРЕБОВАНИЯ К ДОКУМЕНТИРОВАНИЮ

Отчётная документация должна передаваться Заказчику в электронном виде на русском языке. Вспомогательная документация (не указанная в качестве непосредственного результата работ) также передаётся только в электронном виде.

7.1 Перечень подлежащих разработке документов

Документы «Проект системы» и «Пояснительная записка» должны разрабатываться согласно требованиям [4].

7.2 Вид представления и количество документов

Нижеперечисленные документы к АС предоставляются в электронном виде в форматах *.docx* и *.pdf* по одному экземпляру каждый

1. Техническое задание;
2. Проект системы;
3. Пояснительная записка;
4. Три варианта дополнительной функциональности на согласование.

7.3 Требования по использованию ЕСКД и ЕСПД при разработке документов

Документы на Систему оформляют в соответствии с требованиями ОС ТУСУР-2021.

Общие требования:

- размер бумаги – А4. Допускается для размещения рисунков и таблиц использование листов формата А3 с подшивкой по короткой стороне листа;
- шрифт – Times New Roman 14;
- первая строка – отступ 1,25 см;
- межстрочный интервал – полуторный;
- выравнивание – по ширине;
- перенос слов – автоматический
- перенос слов из прописных букв – отменить.

8 ИСТОЧНИКИ РАЗРАБОТКИ

В настоящем документе использованы следующая литература и нормативные документы:

1. Новые технологии в программировании : учебное пособие / А. А. Калентьев, Д. В. Гарайс, А. Е. Горяинов — Томск : Эль Контент, 2014. — 176 с.
2. ГОСТ 34.603 «Информационная технология. Виды испытаний автоматизированных систем»
3. ГОСТ 34.602 – 2020 «Информационные технологии. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы»;
4. ОС ТУСУР 01-2021 «Работы студенческие по направлениям подготовки и специальностям технического профиля. Общие требования и правила оформления от 25.11.2021»;
5. Рабочая программа дисциплины «Основы разработки САПР»;
6. Учебное пособие для студентов направления «Электроника и микроэлектроника» «Математические модели и САПР электронных приборов и устройств»;
7. Введение в UML от создателей языка [Текст] : руководство пользователя / Г. Буч, Д. Рамбо, И. Якобсон. - 2-е изд. - М. : ДМК Пресс, 2012. - 494 с. : ил. - (Классика программирования). - Предм. указ.: с. 483-493. - ISBN 978-5-94074-644-7;
8. Ли. К. Основы САПР (CAD/CAM/CAE). – Спб.:«Питер», 2004. – 560с.

Приложение Б
(обязательное)

Пример титульного листа

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждения
высшего образования

**ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ СИСТЕМ
УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)**

Кафедра компьютерных систем в управлении и проектировании (КСУП)

РАЗРАБОТКА ПЛАГИНА «КНИЖНЫЙ ШКАФ»

ДЛЯ «КОМПАС-3D»

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

по дисциплине

«Основы разработки САПР» (ОРСАПР)

Выполнил:

студент гр. 580-3

_____ Иванов И.И.

« ____ » _____ 2023 г.

Руководитель:

к.т.н., доцент каф. КСУП

_____ Калентьев А.А.

« ____ » _____ 2023 г.

Томск, 2023

Приложение В

(обязательное)

Стандарт оформления кода

1. Введение

Этот документ описывает единый стиль кода, используемый в компании «50ом Технолоджиз». Стиль ориентирован на разработку на языке C#, но большинство из них применимо ко всем компилируемым языкам с C-подобным синтаксисом. В основном данный стандарт базируется на стандарте RSDN, с некоторыми уточнениями и дополнениями. Основной текст можно найти на официальном сайте RSDN[22].

Помните! Код чаще читается, чем пишется, поэтому не экономьте на понятности и чистоте кода ради скорости набора.

2. Список терминов

В русском языке есть некоторые разночтения в названии некоторых частей программы. Во избежание путаницы в этом разделе будут приведены часто встречающиеся термины, их английские аналоги и краткое пояснение.

- **operator** – оператор (например, +, -, *, !)
- **statement** – инструкция (например, «a = b;», «if (a) {}»)
- **expression** – выражение (например, «a – b + c», «a == 1»)

Также далее по тексту под термином "имя" подразумевается имя любого идентификатора - переменной, метода, класса, пространства имен и т.п., если нет дополнительного уточнения

3. Именованное

3.1 Общие правила

Пользоваться регионами запрещено!

3.2 Стили именования

Существует несколько стилей именования

1. **Pascal**

- Первая буква каждого слова заглавная, остальные строчные
- Разделители между слов не используются
- Примеры: Color, BackColor, LastModified, DateTime

2. **Camel**

- Первая буква первого слова строчная, остальные первые буквы каждого нового слова - заглавные

- Разделители между слов не используются
- Примеры: color, backColor, lastModified, dateTime

3. **Snake (запрещен)**

- Все буквы строчные
- Между словами используется разделитель '_' или '-'
- Примеры: color, back_color, last_modified, date_time

Используются только стили Pascal и Camel. Есть несколько исключений, которые будут перечислены ниже, например, именование закрытых полей, именование обработчиков событий в формах Windows Forms, именование юнит-тестов. Не используйте малопонятные префиксы или суффиксы (например, венгерскую нотацию), современные языки и средства разработки позволяют контролировать типы данных на этапе разработки и сборки.

Табуляция и оформление фигурных скобок

Вместо табуляции используется 4 пробела. Далее в тексте слово "табуляции" используется в контексте "4 пробела". В подавляющем большинстве случаев используется стиль Stallman:

- Открывающаяся фигурная скобка переносится на следующую строку без сдвига относительно предыдущей, единственный символ в строке

– Закрывающаяся фигурная скобка находится вертикально под открывающейся скобкой, единственный символ в строке

– Содержимое фигурных скобок сдвигается на одну табуляцию

Пример:

```
if (condition)
{
    Console.WriteLine("Hello, World!");
}
```

В отдельных случаях фигурные скобки могут оформляться иначе (автосвойства и т.п.) Примеры исключений будут указаны отдельно ниже.

4 Стил ь оформления

4.1 Оформление

1. Знак табуляции не используем. Используем 4 пробела вместо табуляции.
2. Избегайте строк длиной больше ширины экрана. Конечно, данный совет зависит от формата экрана и настроек IDE, но это примерно 100 символов в строку.
3. При переносе части кода инструкций и описаний на другую строку вторая и последующая строки должны быть отбиты вправо на один отступ (табуляцию). Если, при переносе строки с параметрами методов, перенесенная строка все еще длиннее 100 символов, то каждый параметр перенести на следующую строку. Пример:

```
public void DoSomething(
    int firstParameter,
    int secondParameter,
    int thirdParameter)
{
    // Do something...
}
public void Main ()
{
    DoSomething(
        firstParameter,
```

```
        secondParameter,  
        thirdParameter);  
    }
```

4. Оставляйте запятую на предыдущей строке так же, как вы это делаете в обычных языках (русском, например).

5. Избегайте лишних скобок, обрамляющих выражения целиком. Лишние скобки усложняют восприятие кода и увеличивают возможность ошибки. Если вы не уверены в приоритете операторов, лучше загляните в соответствующий раздел документации.

6. Не размещайте несколько инструкций на одной строке. Каждая инструкция должна начинаться с новой строки.

4.2 Linq-запросы

1. Все идентификаторы в запросе должны быть читаемы и их назначение должно быть понятно. При переносе длинных запросов на другую строку применяется вид оформления: если часть запроса переносится, то на следующей строке запрос продолжается с отступлением на одну табуляцию.

```
var localDistributors = from customer in customers  
    join distributor in distributors on customer.City  
    equals distributor.City  
    select new { Customer = customer, Distributor =  
distributor };
```

2. Разделять запросы на строки лучше так, чтобы новая строка начиналась со слов `from`, `where`, `join`, `select`. Для доступа к внутренним коллекциям вместо предложения `join` используйте несколько предложений `from`. Например, коллекция объектов `Student` может содержать коллекцию результатов тестирования. При выполнении следующего запроса возвращаются результаты, превышающие 90 баллов, а также фамилии учащихся, получивших такие оценки.

```
var scoreQuery = from student in students  
    from score in student.Scores  
    where score > 90
```

```
select new { Last = student.LastName, score };
```

3. При использовании запросов в виде методов расширения, при переносе частей запросов, строки необходимо начинать с точек.

```
var    scoreQuery    =    students.Select(student    =>  
student.Score)  
    .Where(score => score > 90)  
    .ToList();
```

4.3 Содержимое классов

Все содержимое классов должно группироваться следующим образом:

1. Вложенные перечисления
2. Вложенные классы
3. Константы
4. Статические поля
5. Поля доступные только для чтения (readonly)
6. Перечисления
7. Внутренние классы
8. Поля
9. Конструкторы
10. Делегаты
11. События
12. Свойства
13. Публичные методы
14. Защищенные методы
15. Приватные методы

Желательно не менять порядок членов, объявленных в интерфейсе при реализации интерфейса

5 Импорт сборок внутри определения пространства имен

На самом деле между ними есть (тонкая) разница. Представьте, что у вас есть следующий код в File1.cs:

```
// File1.cs
using System;
namespace Outer.Inner
{
    class Foo
    {
        static void Bar()
        {
            double d = Math.PI;
        }
    }
}
```

Теперь представьте, что кто-то добавляет в проект еще один файл (File2.cs), который выглядит так:

```
// File2.cs
namespace Outer
{
    class Math
    {
        ...
    }
}
```

Компилятор ищет Outer, прежде чем искать те, которые используют операторы за пределами пространства имен, поэтому он находит Outer.Math вместо System.Math. К сожалению (или, может быть, к счастью?), Outer.Math не имеет члена PI, поэтому File1 теперь поврежден.

Это изменится, если вы поместите использование внутри объявления пространства имен следующим образом:

```
// File1b.cs
namespace Outer.Inner
{
    using System;

    class Foo
```



```
    {  
        static void Bar()  
        {  
            double d = Math.PI;  
        }  
    }  
}
```

Теперь компилятор ищет System перед поиском Outer, находит System.Math, и все в порядке. Кто-то может возразить, что Math может быть плохим названием для определяемого пользователем класса, поскольку такой класс уже есть в System; дело тут как раз в том, что разница есть, и она влияет на ремонтпригодность вашего кода. Также интересно отметить, что происходит, если Foo находится в пространстве имен Outer, а не в Outer.Inner. В этом случае добавление Outer.Math в File2 разбивает File1 независимо от того, куда идет использование. Это означает, что компилятор просматривает самое внутреннее объемлющее пространство имен, прежде чем он просматривает любые операторы использования.

Приложение Г (обязательное)

Руководство по настройке инструментов разработки

Актуально для версии Visual Studio 2022 Version 17.8.6, ReSharper версии 2023.2, Editor Guidelines версии 2.2.11, XAML Styler версии 3.22.08.1 и Visual Studio Spell Checker версии 2023.12.29.0.

1 Настройка Visual Studio. Импорт настроек Visual Studio.

1. Нужно нажать на панели инструментов «Tools->Import and Export Settings...»(рисунок Г.1).

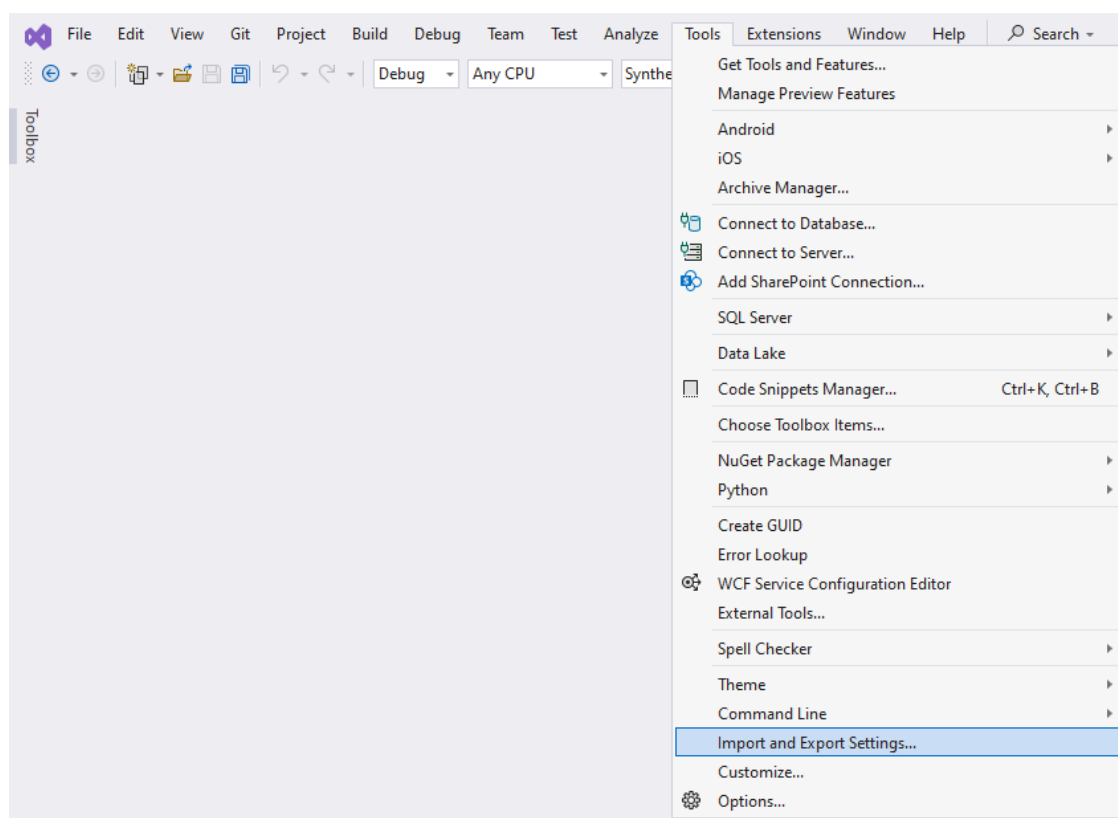


Рисунок Г.1 – Вызов «Tools->Import and Export Settings...»

2. Далее выбираем пункт «Import selected environment settings» и нажимаем «Next» (рисунок Г.2).

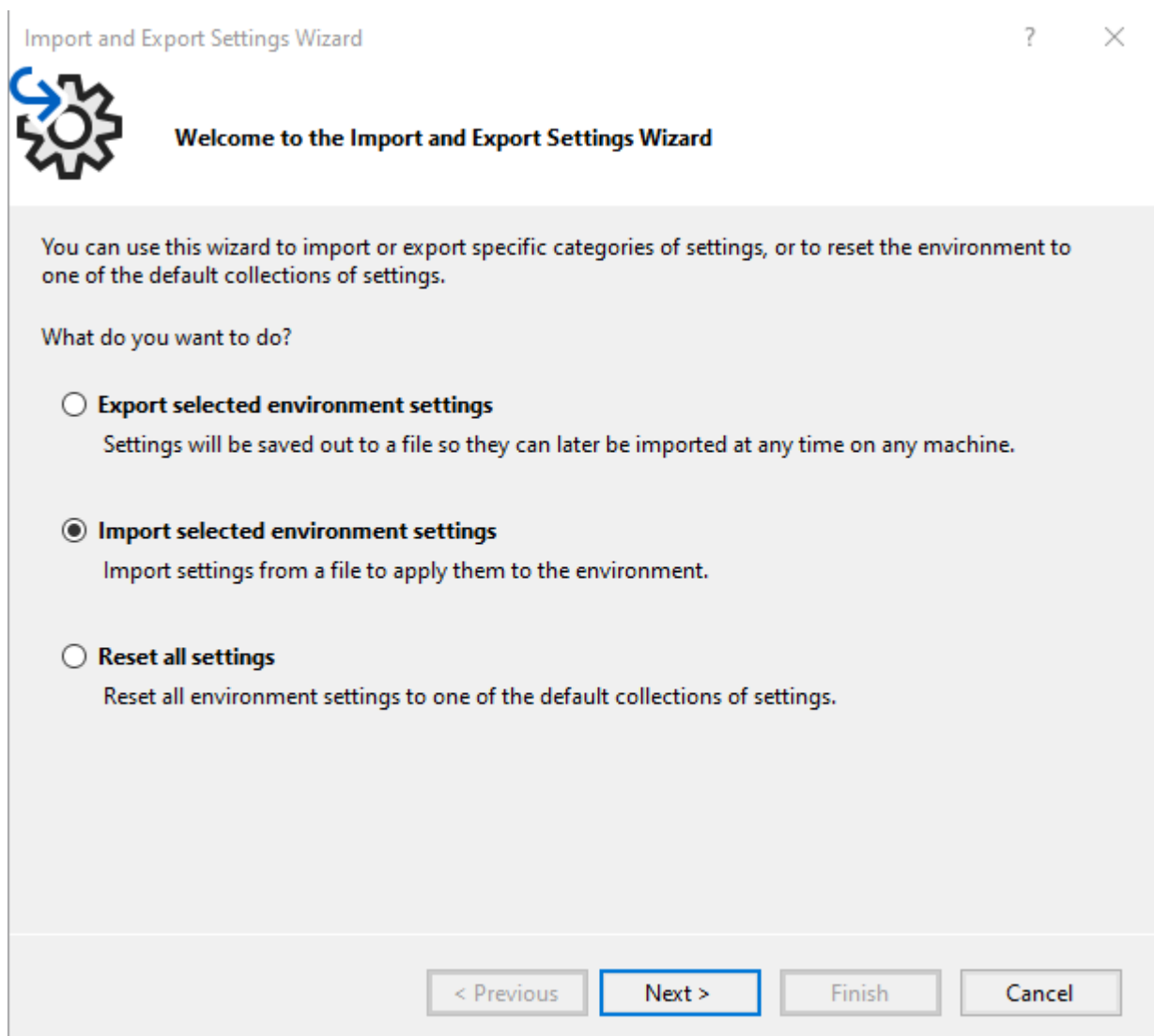


Рисунок Г.2 – Окно «Import and Export Settings Wizard»

3. Если нужно сохраняем текущий файл конфигурации (рисунок Г.3). Также можно пропустить шаг (второй пункт на окне).

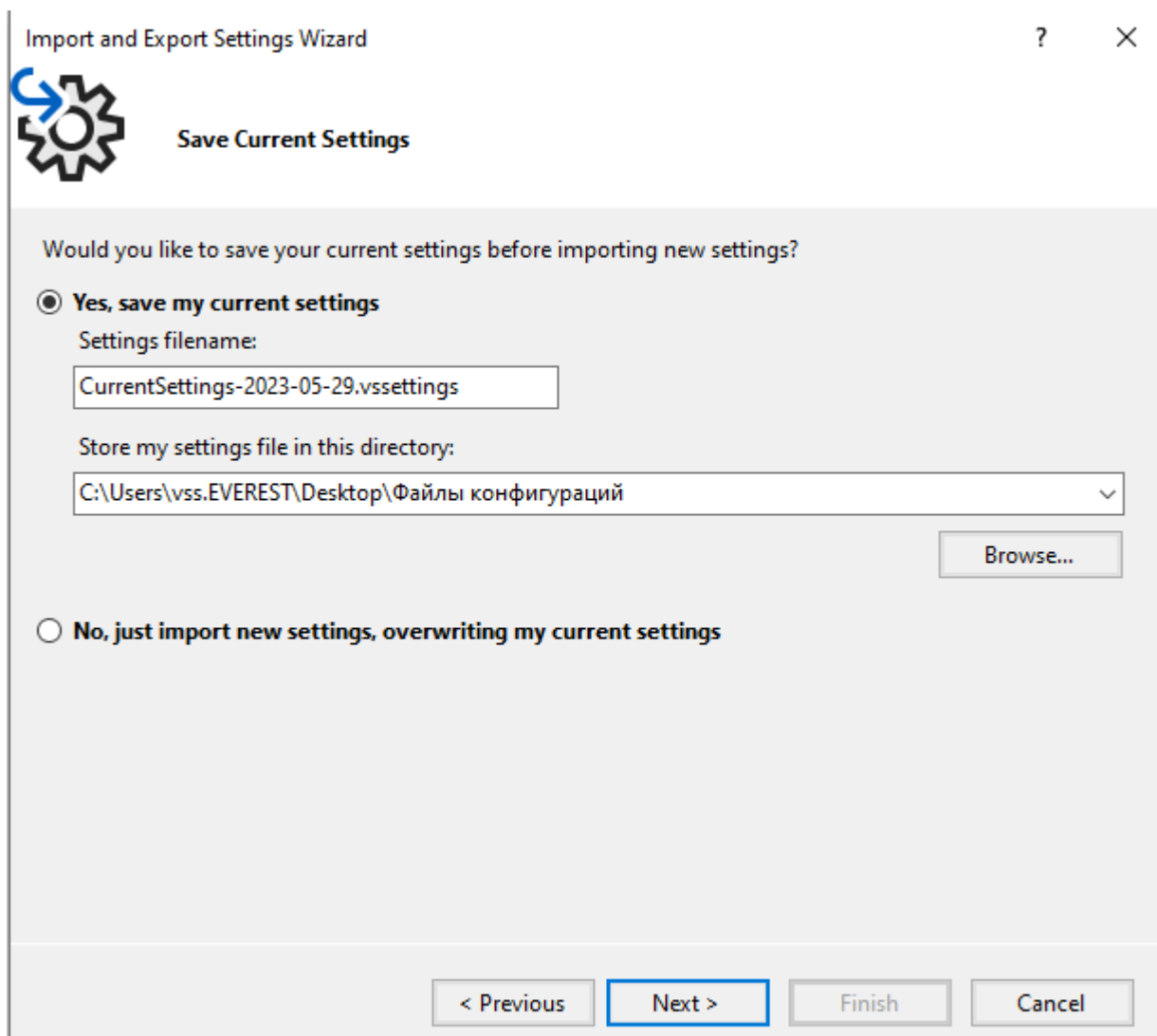


Рисунок Г.3 – Окно «Save Current Settings»

4. После чего нажимаем на кнопку «Browse...», чтобы выбрать файл конфигурации (рисунок Г.4).

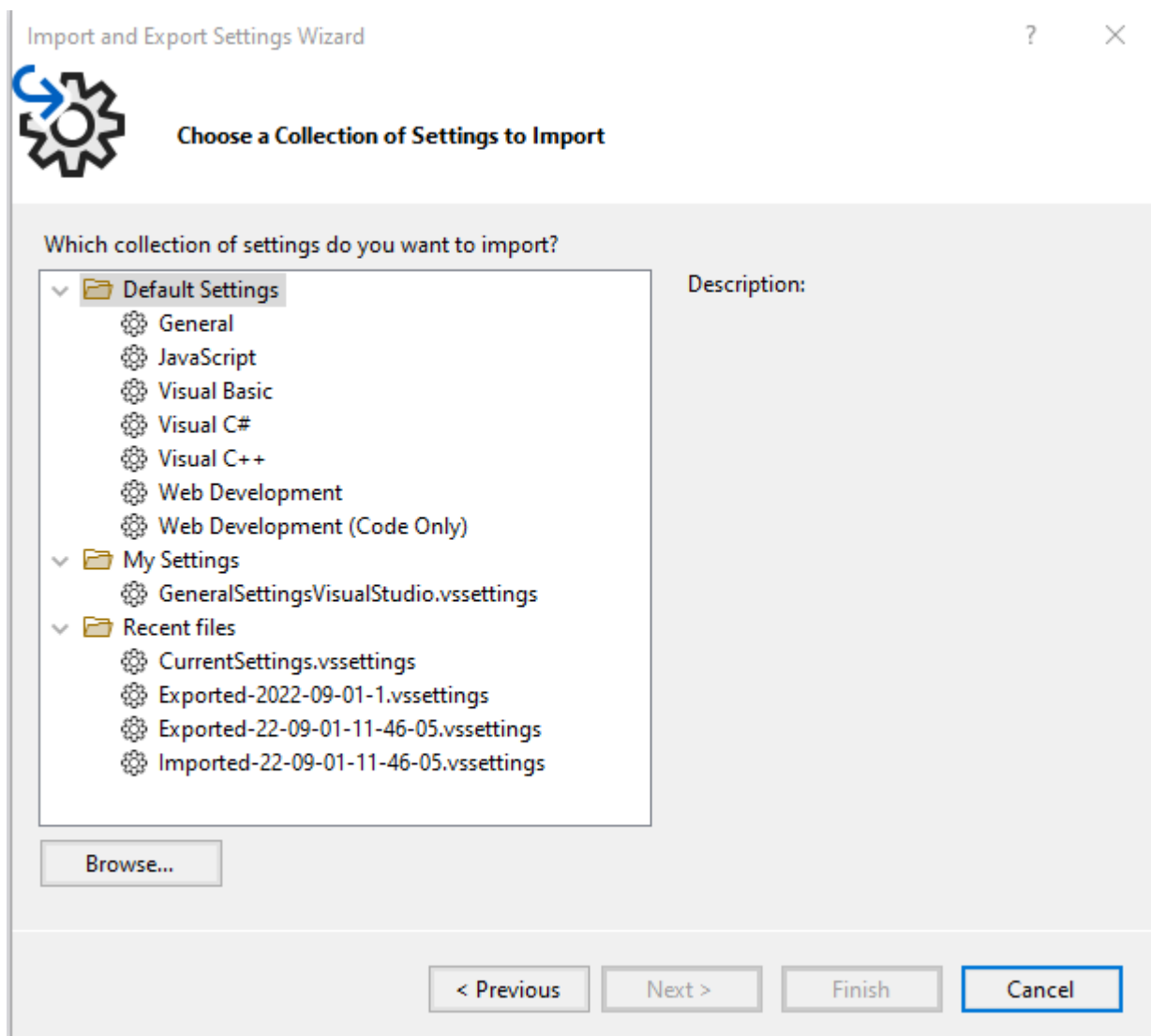


Рисунок Г.4 – Окно «Choose a Collection of Settings to Import»

5. Переходим в нужный каталог, выбираем файл (рисунок Г.5) и нажимаем «Next».

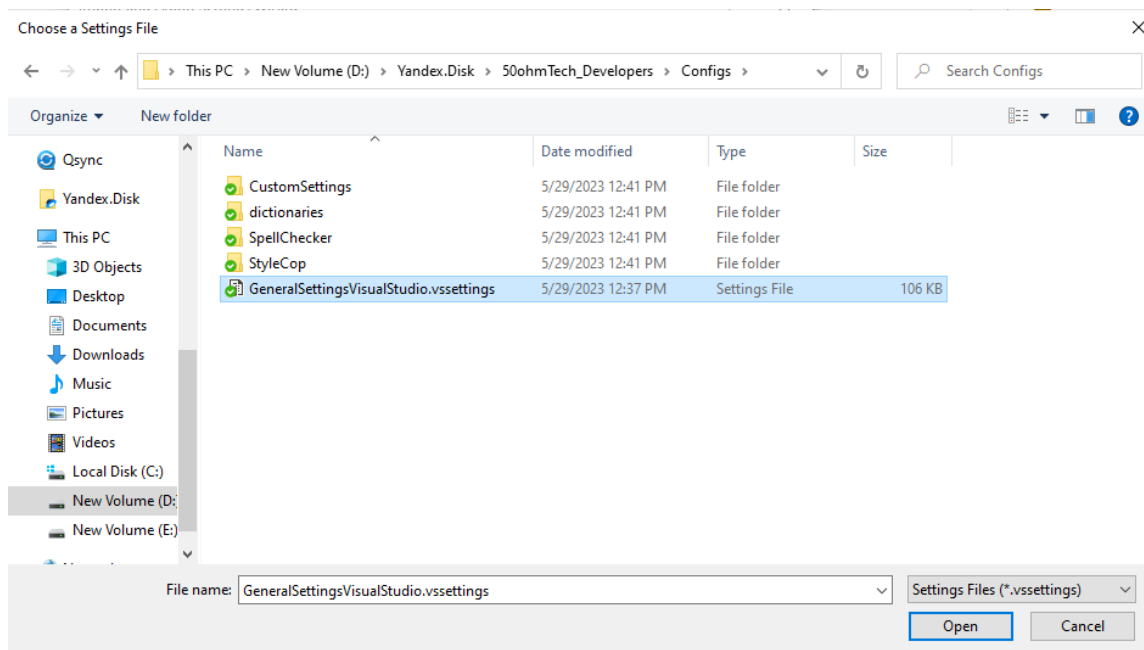


Рисунок Г.5 – Окно выбора настроек

6. Выбираем нужные настройки из файла конфигурации, нажимаем на «Finish» (рисунок Г.6) и закрываем окно.

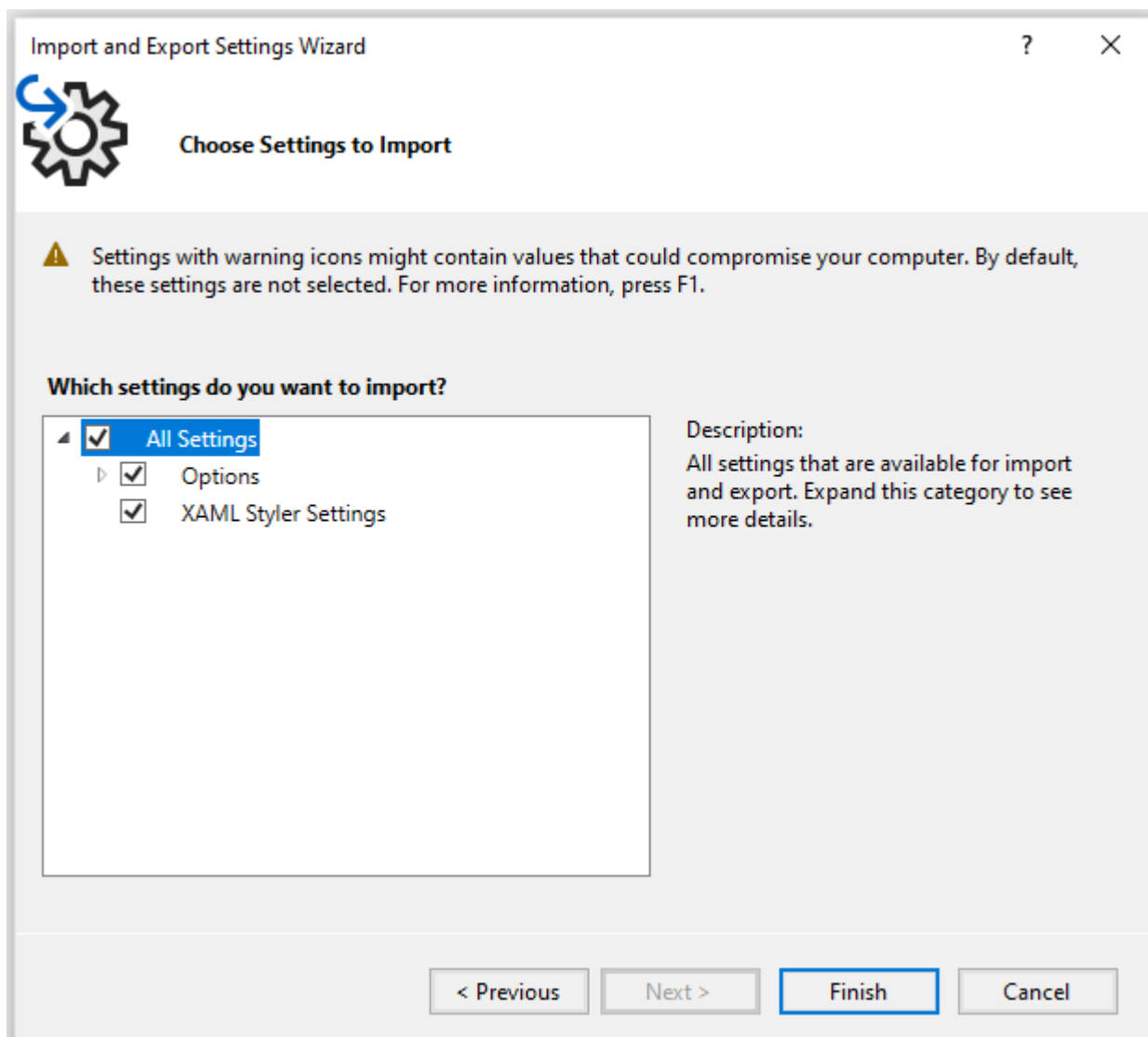


Рисунок Г.6 – Окно «Choose Settings to Import»

2 Настройка ReSharper. Импорт настроек ReSharper.

1. Нужно нажать на панели инструментов «Extensions->ReSharper->Options» (рисунок Г.7).

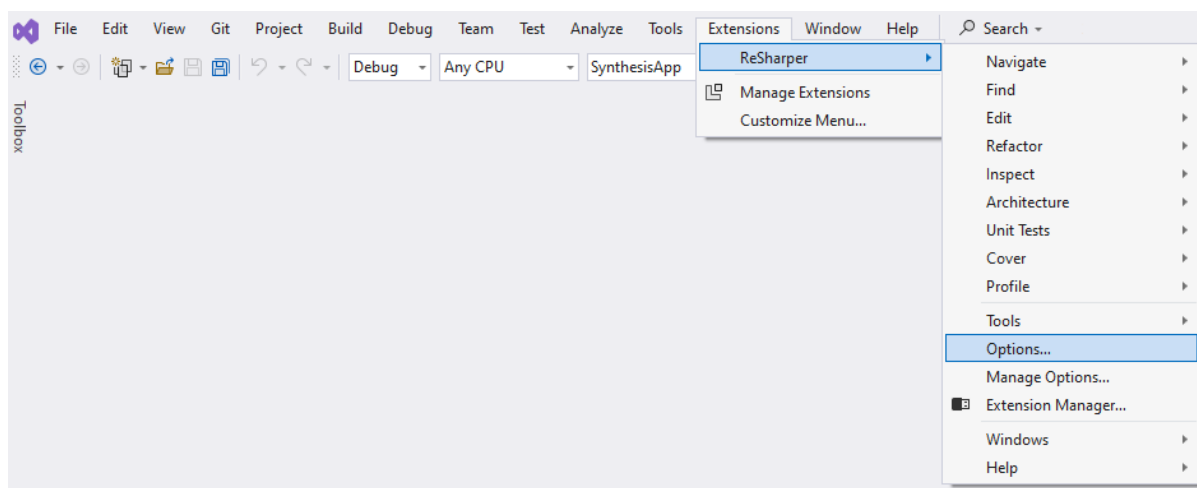


Рисунок Г.7 – Открытие настроек для ReSharper

2. Нажимаем на кнопку «Manage...» (рисунок Г.8, снизу слева)

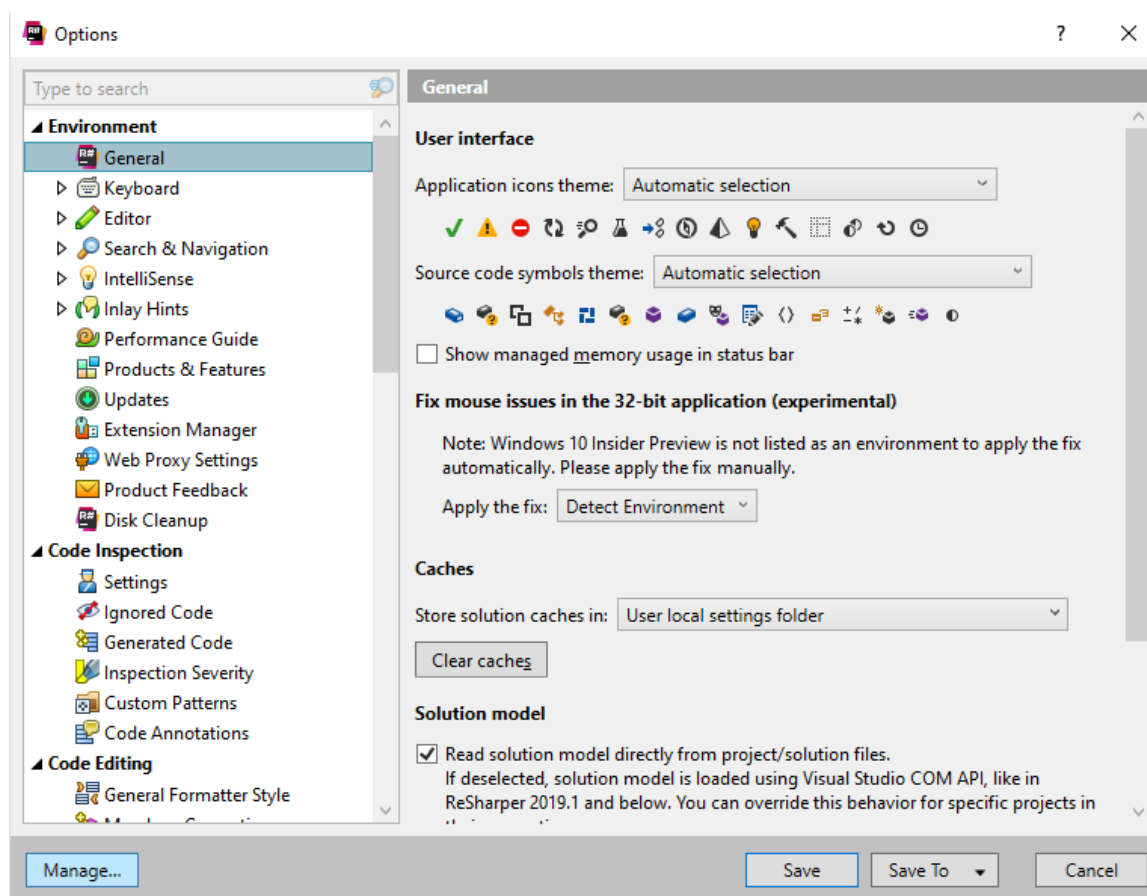


Рисунок Г.8 – Окно «Option»

3. Тут есть два варианта: добавить дополнительный файл настроек или импортировать в основную конфигурацию настройки.

2.1 Добавление дополнительного файла настроек

1. Если нужно добавить к текущим настройкам файл конфигурации, то рядом с «This computer» нажмите на значок «+», а затем на «Open Settings File...» (рисунок Г.9).

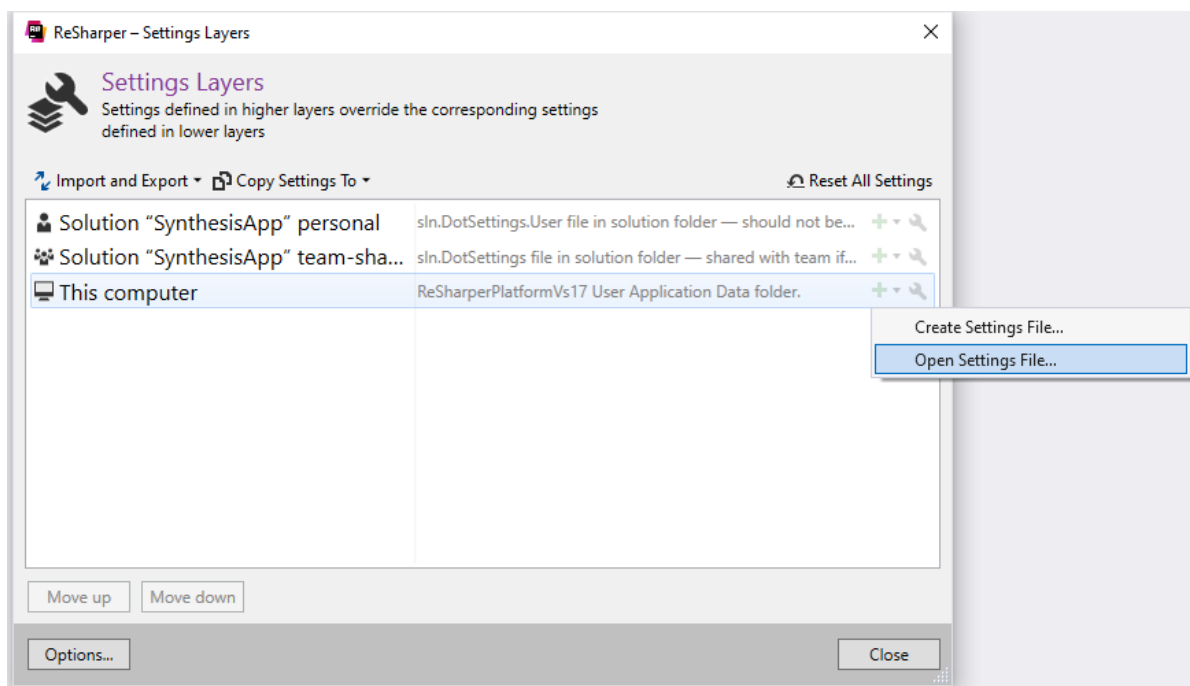


Рисунок Г.9 – Окно выбора установки настроек ReSharper

2. Выберите файл (рисунок Г.10).

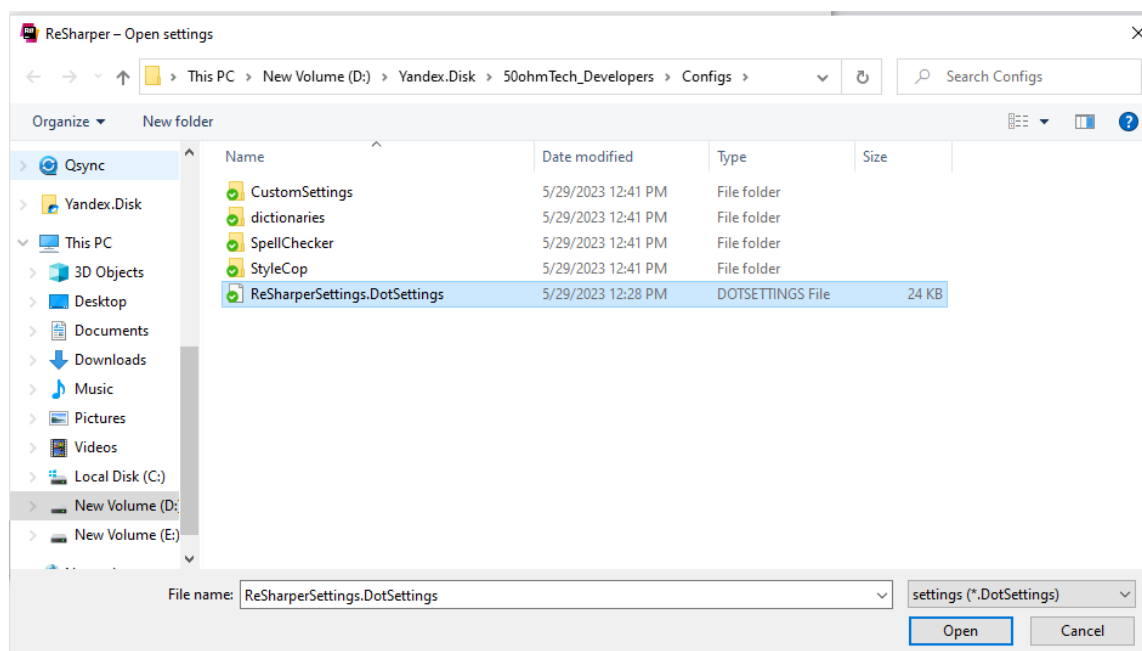


Рисунок Г.10 – Выбор файла настроек

3. Тогда под «This computer» появится загруженная конфигурация (рисунок Г.11).

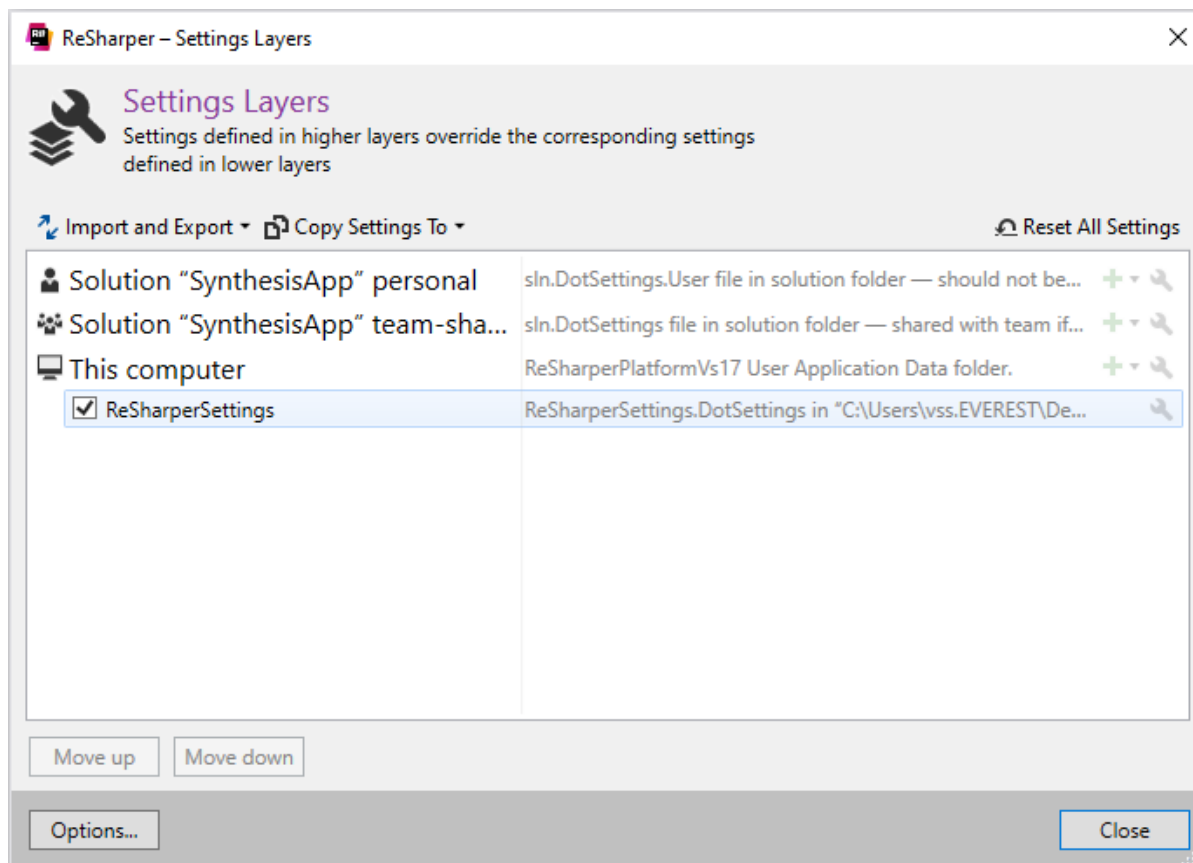


Рисунок Г.11 – Результат экспорта настроек

2.2 Импорт в основную конфигурацию настроек

1. Если нужно импортировать настройки в основную конфигурацию, то нужно нажать правой кнопкой мыши на «This computer», затем «Import from->Import from File».

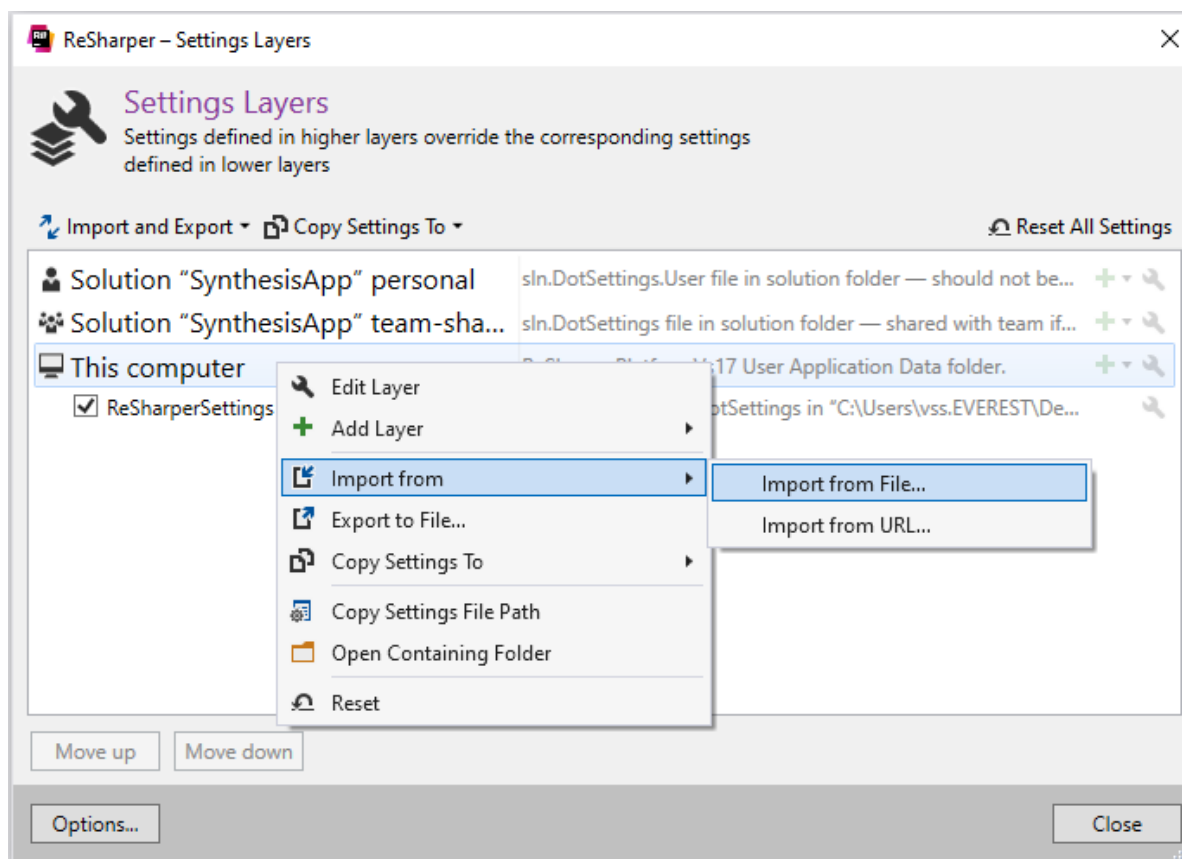


Рисунок Г.12 – Импорт в основную конфигурацию настроек

2. Выбираем файл (см. рисунок Г.10).
3. Выбираем настройки, которые нужно импортировать и нажимаем «ОК» (рисунок Г.13).

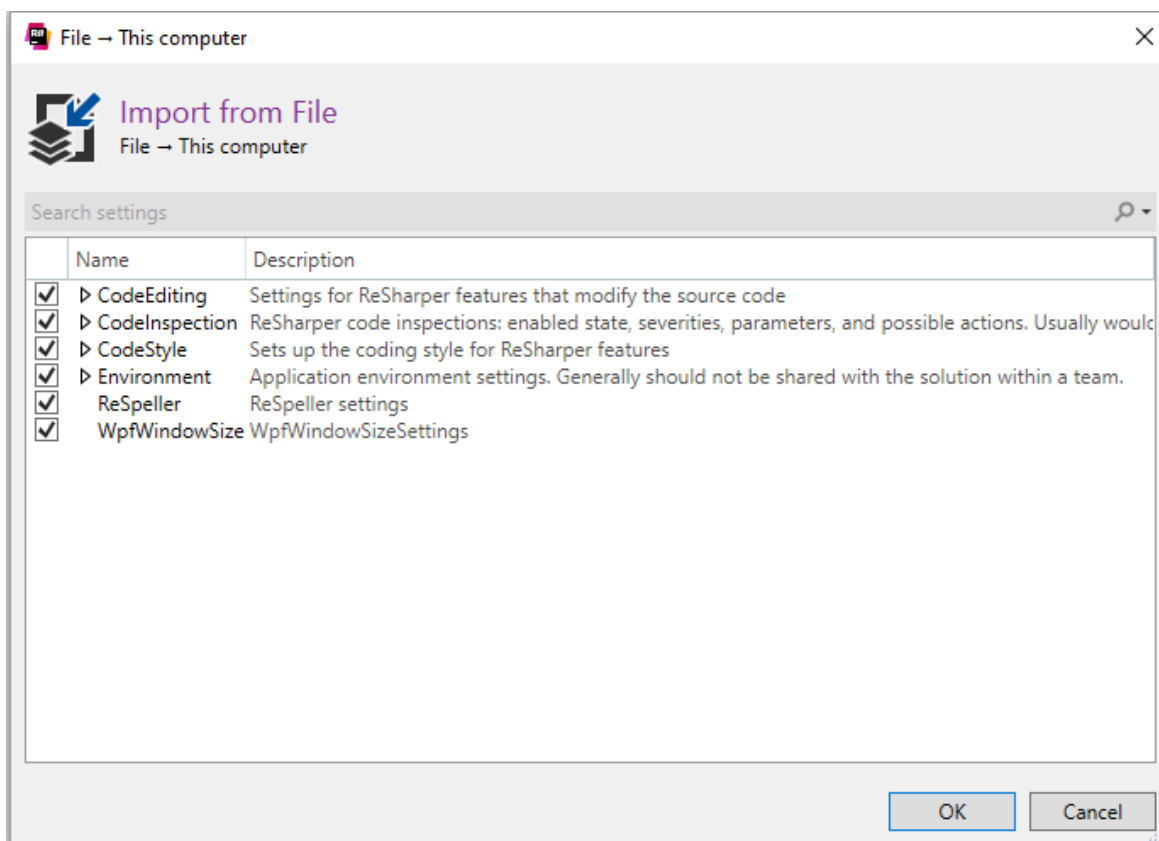


Рисунок Г.13 – Выбор нужных настроек конфигурации ReSharper

3 Добавление словарей для ReSpeller

1. Нужно нажать на панели инструментов Extensions->ReSharper->Options (см. рисунок Г.7).
2. Затем находим ReSpeller (находится в секции Tools) (рисунок Г.14).

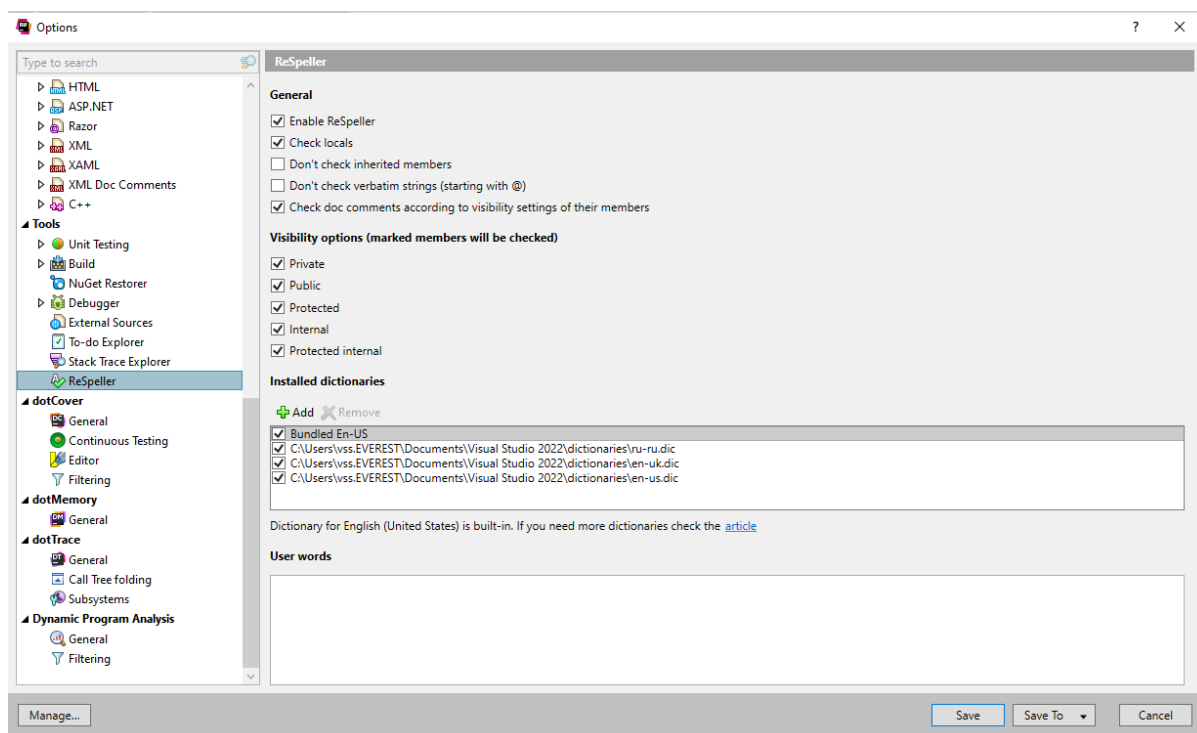


Рисунок Г.14 – Вкладка «ReSpeller»

3. Нажимаем на кнопку «Add» и выбираем нужный словарь. Важно чтобы был файл с расширением «.dic», а также файл с таким же именем с расширением «.aff» (рисунок Г.15).

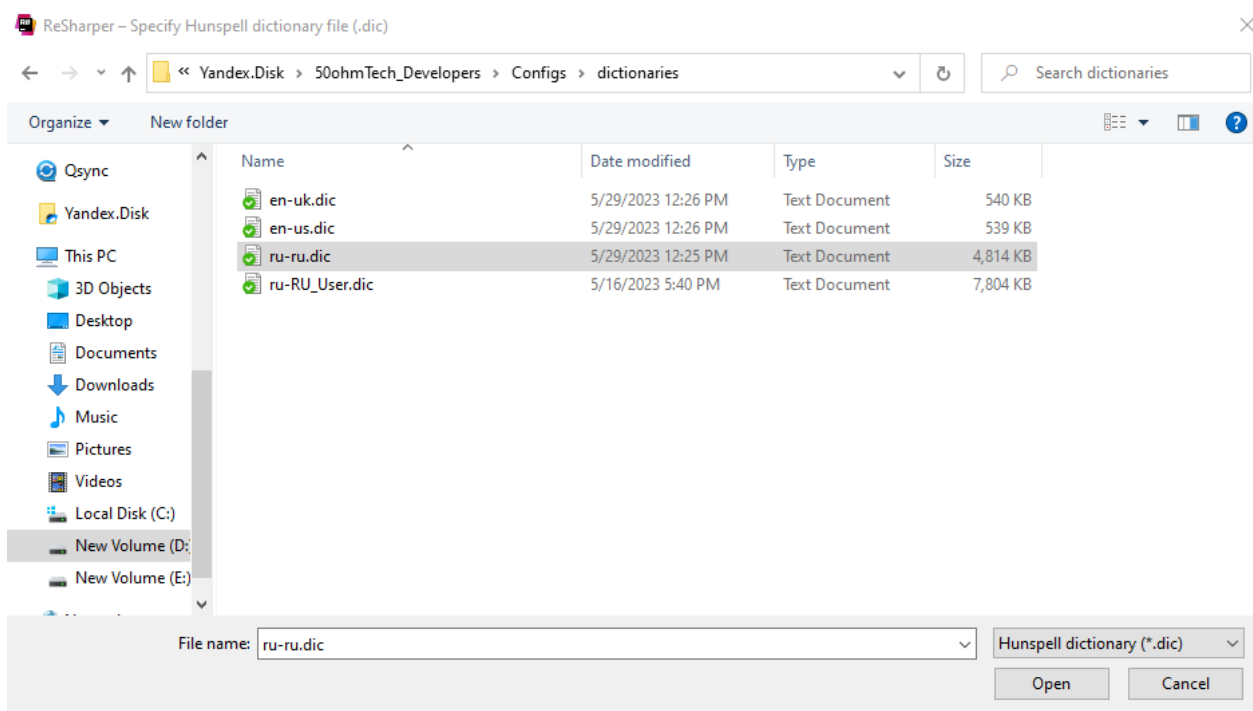


Рисунок Г.15 – Окно с выбором словарей

4 Импорт библиотек в Visual Studio Spell Checker.

Альтернативным инструментом для проверки орфографии в Visual Studio является расширение Visual Studio Spell Checker. Его преимущество перед Spell Checker от ReSharper в том, что орфография будет проверяться даже в поле ввода сообщения коммита.

1. Visual Studio Spell Checker можно установить отдельно через «Extensions -> Manage Extensions» (рисунок Г.16).

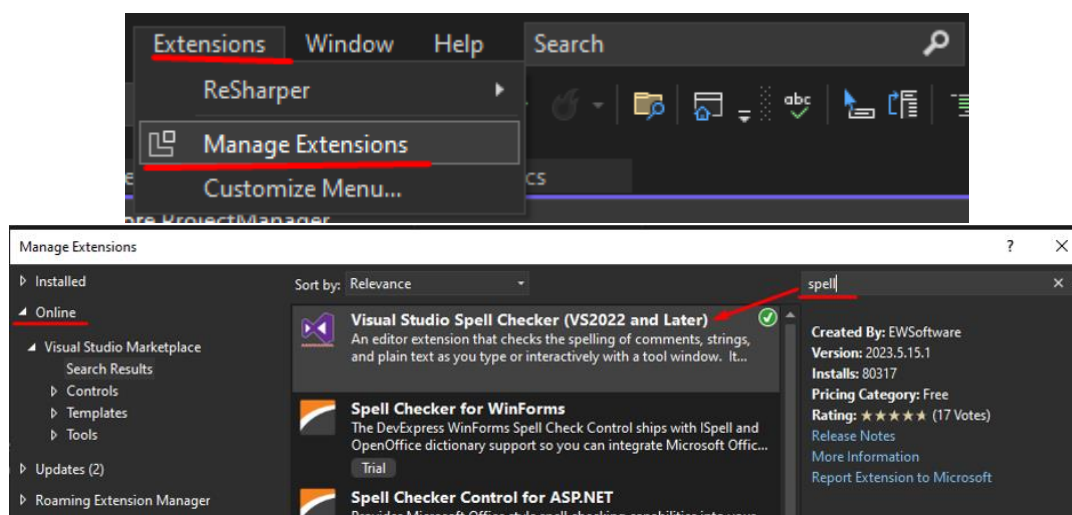


Рисунок Г.16 – Установка Visual Studio Spell Checker

2. Для открытия окна настроек и загрузки словарей нужно нажать на панели инструментов «Tools -> Spell Checker -> Edit Global Configuration» (рисунок Г.17).

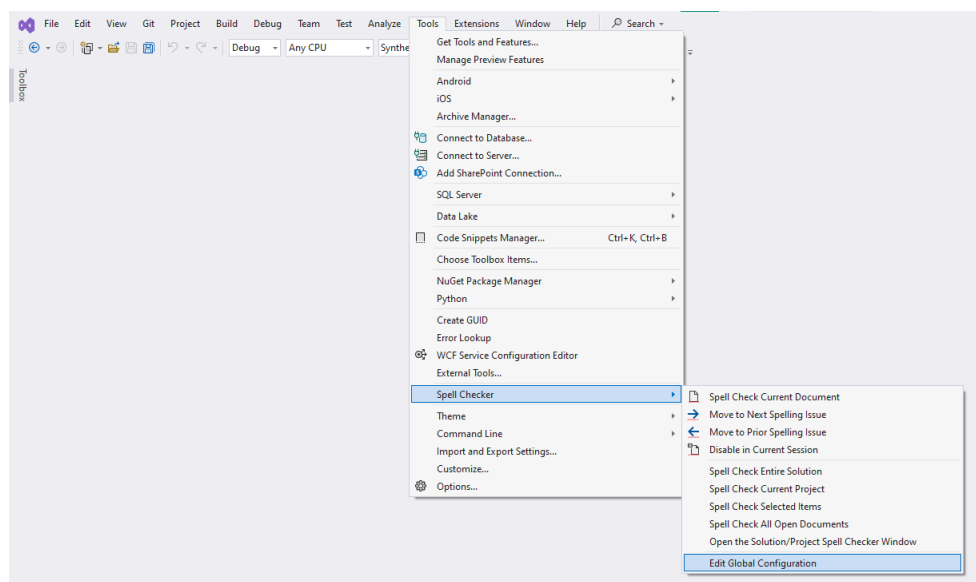


Рисунок Г.17 – Открытие настроек Spell Checker

3. Выбираем в списке Dictionary Settings, затем выбираем нужный язык в комбобоксе Language(s) и добавляем его нажатием кнопки Add. Для добавления словаря нажимаем на кнопку Import (рисунок Г.18).

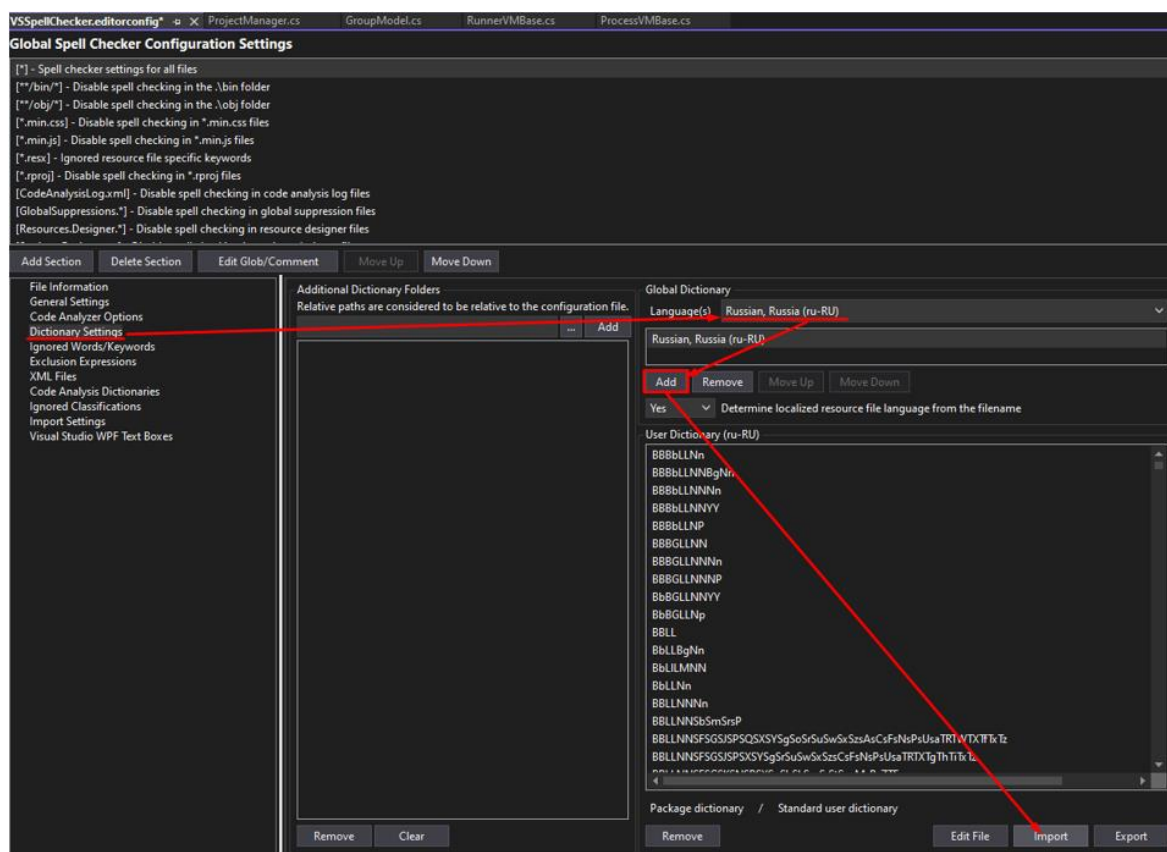


Рисунок Г.18 – Окно настроек Spell Checker

4. Выбираем нужный словарь (см. рисунок Г.15).

5. Для одного языка можно импортировать несколько файлов словарей. Для этого еще раз нажимаем на кнопку Import, выбираем другой файл словаря. Далее будет показан диалог с двумя вариантами: полностью заменить текущий словарь на новый (вариант Yes) или слить новый словарь с существующим (вариант No). Нам нужно использовать слияние (рисунок Г.19).

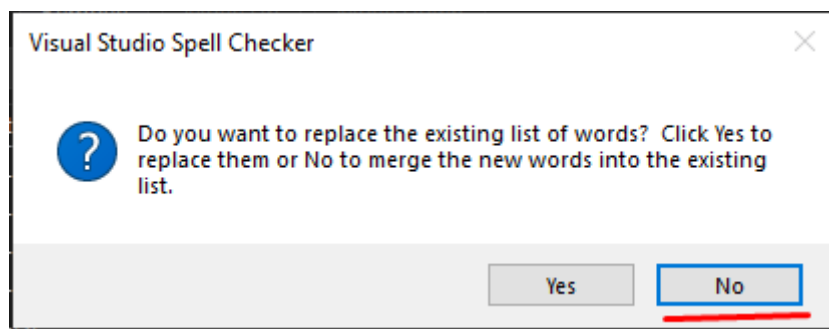


Рисунок Г.19 – Окно с перезаписью старых слов в словаре

6. Необходимо повторить процедуру добавления словарей для ВСЕХ нужных языков, включая английские. После сохранить изменения в файле VSSpellChecker.editorconfig при его закрытии.

7. Перезагрузить Visual Studio.

8. Так как Visual Studio Spell Checker выполняет те же функции, что и ReSharper Spell Checker, что-то одно из них можно отключить. Например, отключение ReSharper Spell Checker (рисунок Г.20).

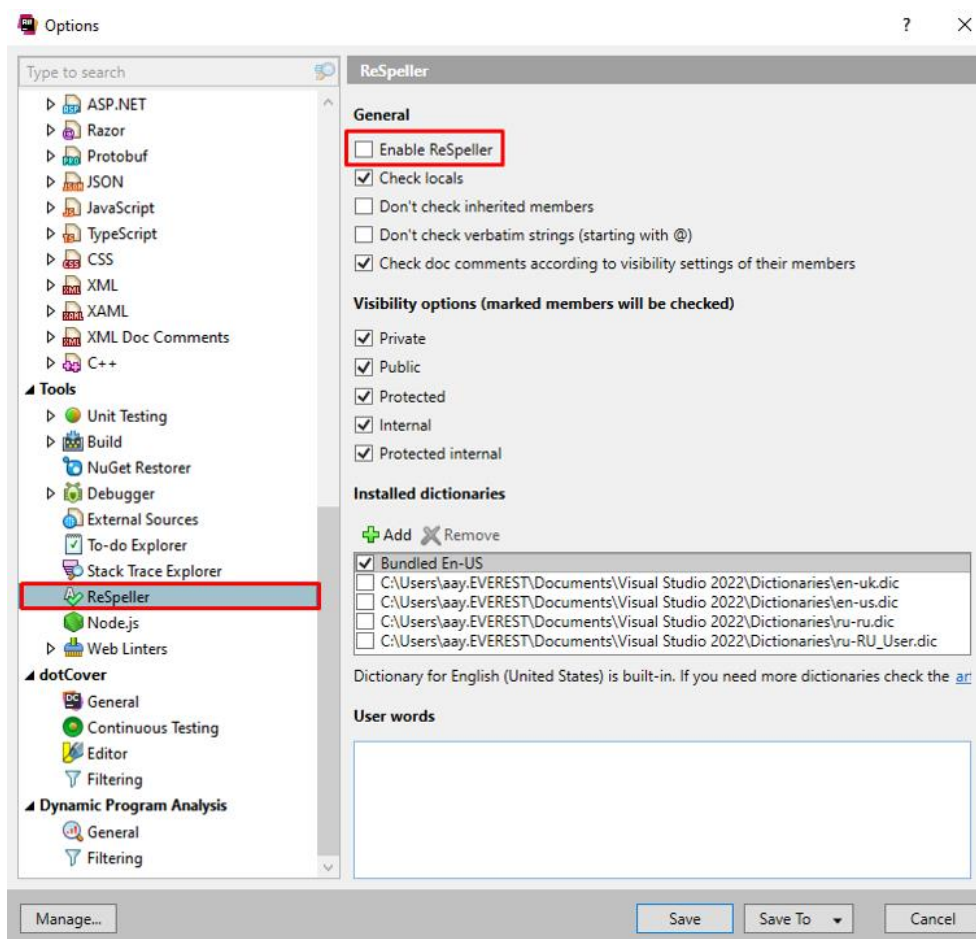


Рисунок Г.20 – Отключение ReSharper

5 Общие настройки XAML Styler.

На рисунке Г.21 представлены настройка для XAML Styler (при использовании WPF).

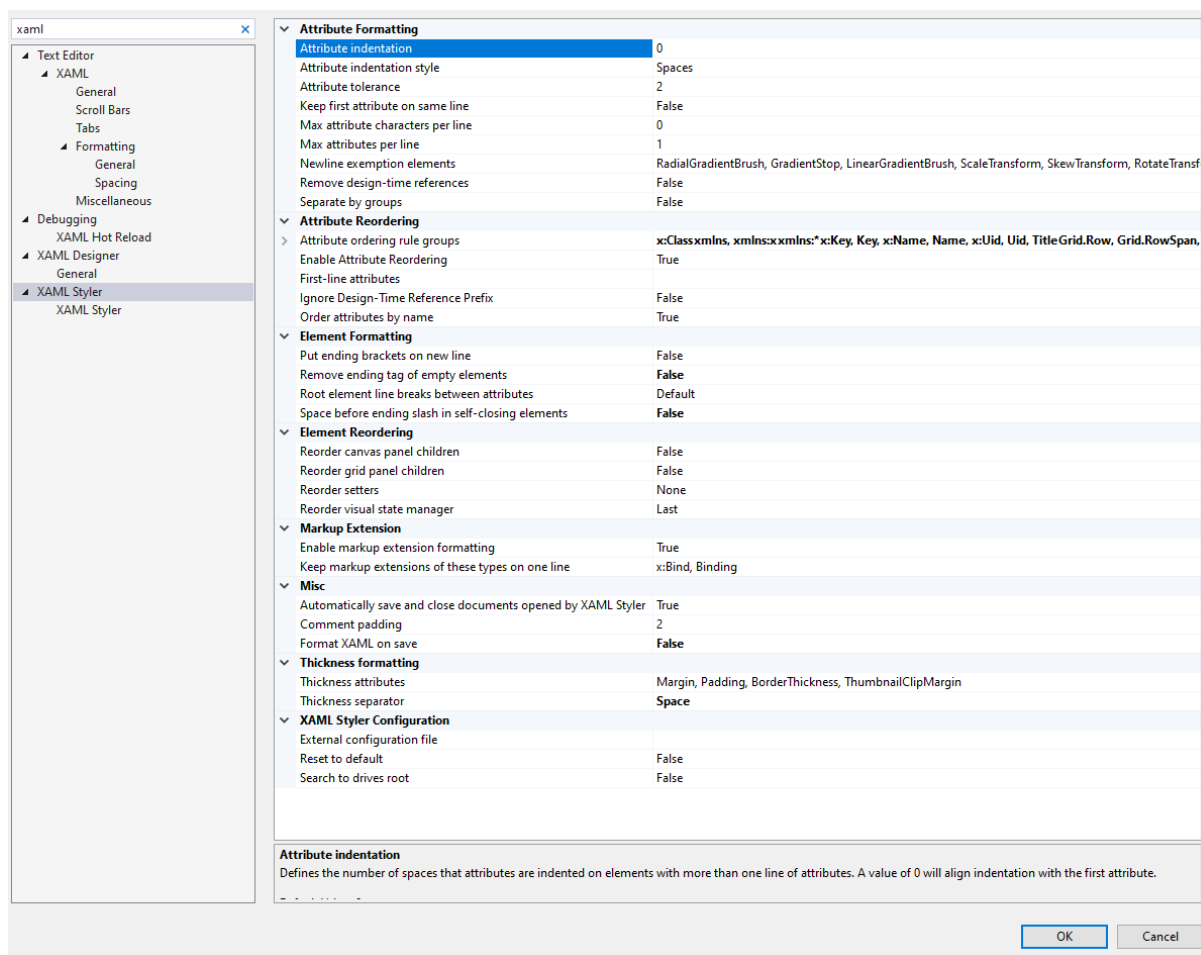


Рисунок Г.21 – Настройки XAML Styler

6 Настройка Editor Guidelines.

Нужно установить две ограничительные линии на 78 и 100 строках, используя расширение Editor Guidelines. За первое ограничение нежелательно заходить, а за вторую линии нельзя – нужно делить такие строки на несколько согласно стандарту оформления кода (приложение В).

Чтобы установить ограничительную линию нужно поставить курсор на нужное место. Затем нажать «ПКМ->Guidelines->Add Guideline».